

ANEXO — CAPTURAS CRÍTICAS DEL CASO DE ESTUDIO

Análisis de Solicitudes 311 — Ciudad de Chicago
Big Data 2026-I

Universidad Externado de Colombia
Departamento de Matemáticas — Programa de Ciencia de Datos

Entidad cliente: City of Chicago — Department of Assets, Information and Services
Dataset: 311 Service Requests · 4.9 GB descomprimido · 13,78 M registros · 39 columnas
Fuente oficial: data.cityofchicago.org/Service-Requests/311-Service-Requests/v6vf-nfxy
Cobertura temporal: diciembre 2018 — abril 2026

Integrantes

Allison Michelle Loango Rayo
Angie Katherine Loango Rayo
Rafael José Orozco Luquez

Fecha de entrega: 26 de mayo de 2026

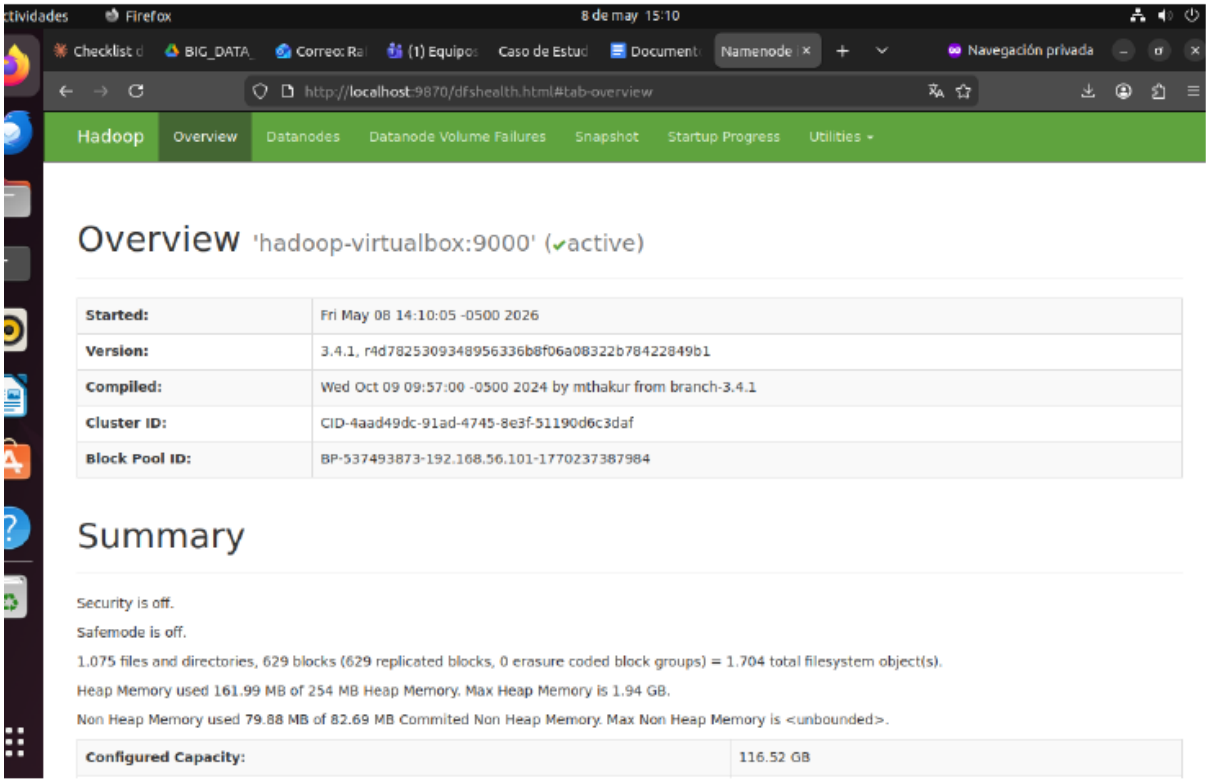
Índice de contenidos

- Sección A** · HDFS Web UI: estado del clúster y bloques (Capturas 1–7)
- Sección B** · YARN Resource Manager: aplicaciones Spark (Capturas 8–9)
- Sección C** · Ejecución del cuaderno, EDA y limpieza (Capturas 10–16)
- Sección D** · Transformaciones, DAGs, planes físicos y salidas materializadas (Capturas 17–23)
- Sección E** · Modelo de Machine Learning (bonificación) (Capturas 24–25)
- Sección F** · Incidentes operativos resueltos (Captura 26)
- Sección G** · Recursos del clúster y optimizaciones (Capturas 27–29)
- Sección H** · Comandos de ejecución documentados (—)
- Sección I** · Declaración obligatoria de uso de IA (—)
- Sección J** · Referencias bibliográficas (APA 7ª edición) (—)

Propósito del documento: este anexo recopila las **29 capturas más importantes** extraídas del informe completo (94 páginas) que evidencian el cumplimiento de los requisitos obligatorios del caso de estudio (reportes de HDFS y YARN, métricas y capturas de Spark UI), junto con los comandos de ejecución documentados, la declaración obligatoria de uso de IA y las referencias bibliográficas en formato APA 7ª edición. Cada captura va acompañada de un pie de página explicativo que describe su contenido técnico y justifica su relevancia respecto a los criterios de evaluación.

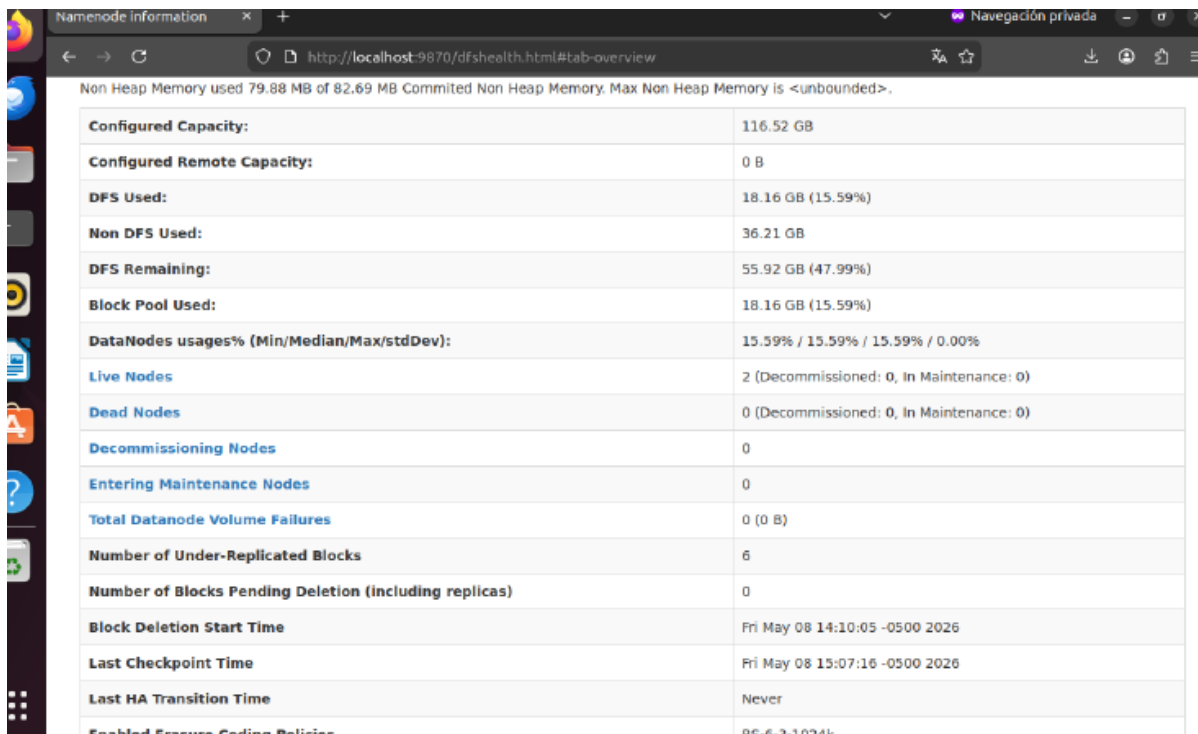
Sección A — HDFS Web UI: estado del clúster y bloques

Captura 1 — HDFS Overview



Pie de página: HDFS Web UI mostrando el estado del NameNode hadoop-virtualbox:9000 en estado active. Se evidencia la versión 3.4.1 de Hadoop, el Cluster ID (CID-4aad49dc-91ad-4745-8e3f-51190d6c3daf), el Block Pool ID (BP-537493873-192.168.56.101-1770237387984) y la Configured Capacity de 116.52 GB. Esta captura cumple el requisito obligatorio del caso de estudio de evidenciar el estado del clúster HDFS.

Captura 2 — HDFS Summary

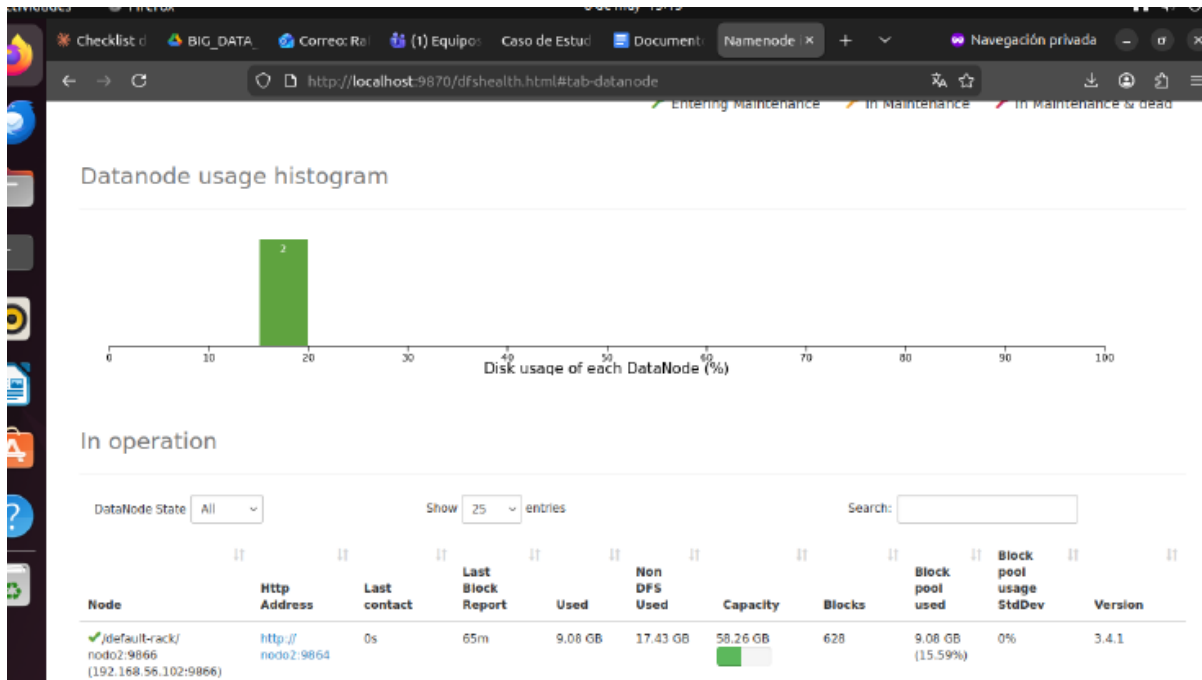


Non Heap Memory used 79.88 MB of 82.69 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	116.52 GB
Configured Remote Capacity:	0 B
DFS Used:	18.16 GB (15.59%)
Non DFS Used:	36.21 GB
DFS Remaining:	55.92 GB (47.99%)
Block Pool Used:	18.16 GB (15.59%)
DataNodes usages% (Min/Median/Max/stdDev):	15.59% / 15.59% / 15.59% / 0.00%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	6
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Fri May 08 14:10:05 -0500 2026
Last Checkpoint Time	Fri May 08 15:07:16 -0500 2026
Last HA Transition Time	Never
Enabled Erasure Coding Policies	RS-6-3-1024k

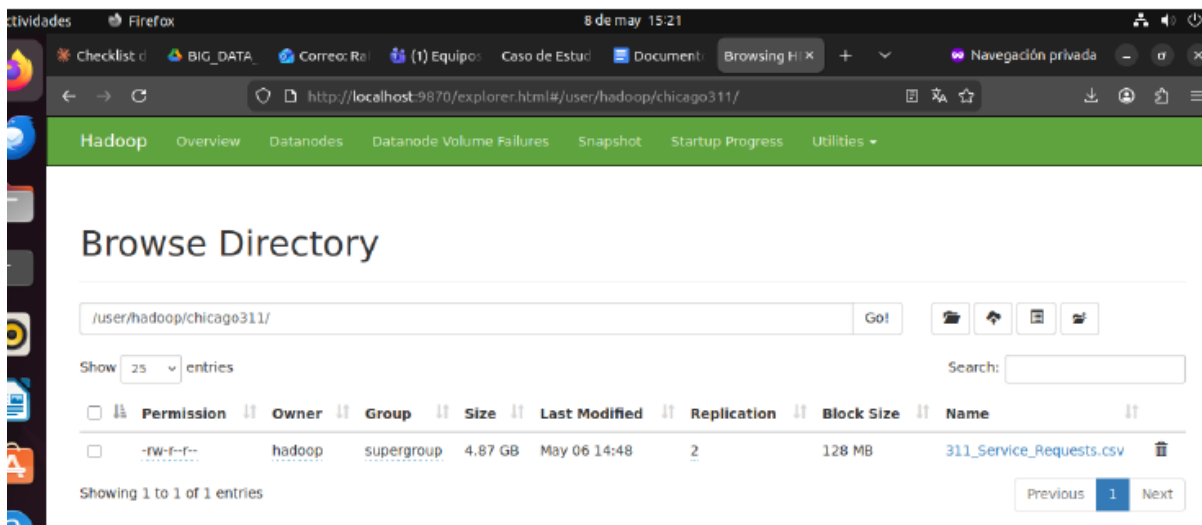
Pie de página: Resumen detallado del estado del sistema de archivos HDFS. Métricas clave: DFS Used 18.16 GB (15.59%), DFS Remaining 55.92 GB (47.99%), Live Nodes 2 y Number of Under-Replicated Blocks = 6 (documentado en el Anexo F como incidente operativo resuelto). Esta captura confirma que el clúster tiene dos DataNodes activos y opera con factor de replicación 2 según los requerimientos del caso.

Captura 3 — DataNodes en operación



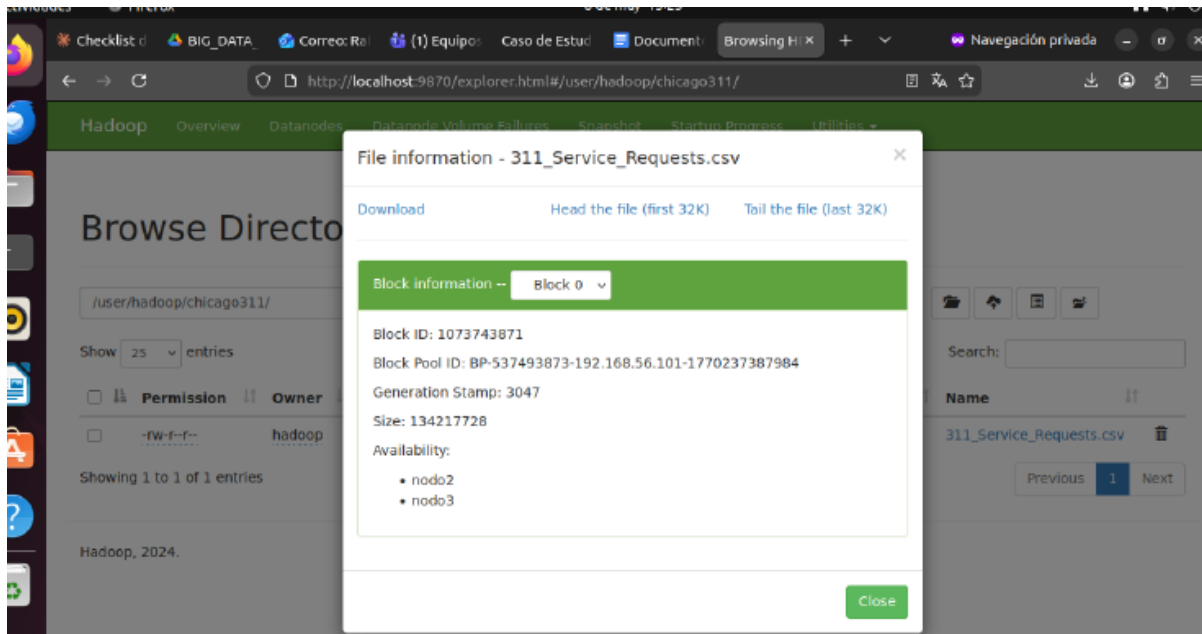
Pie de página: Listado de DataNodes activos: nodo2 (192.168.56.102:9866) y nodo3 (192.168.56.103:9866), ambos con 9.08 GB usados (15.59% de capacidad) y 628 bloques cada uno. La distribución uniforme entre los dos nodos confirma que el factor de replicación 2 está aplicado correctamente y que los bloques del archivo 311_Service_Requests.csv están repartidos entre ambos workers.

Captura 4 — Browse Directory /user/hadoop/chicago311/ cap3 y 4:



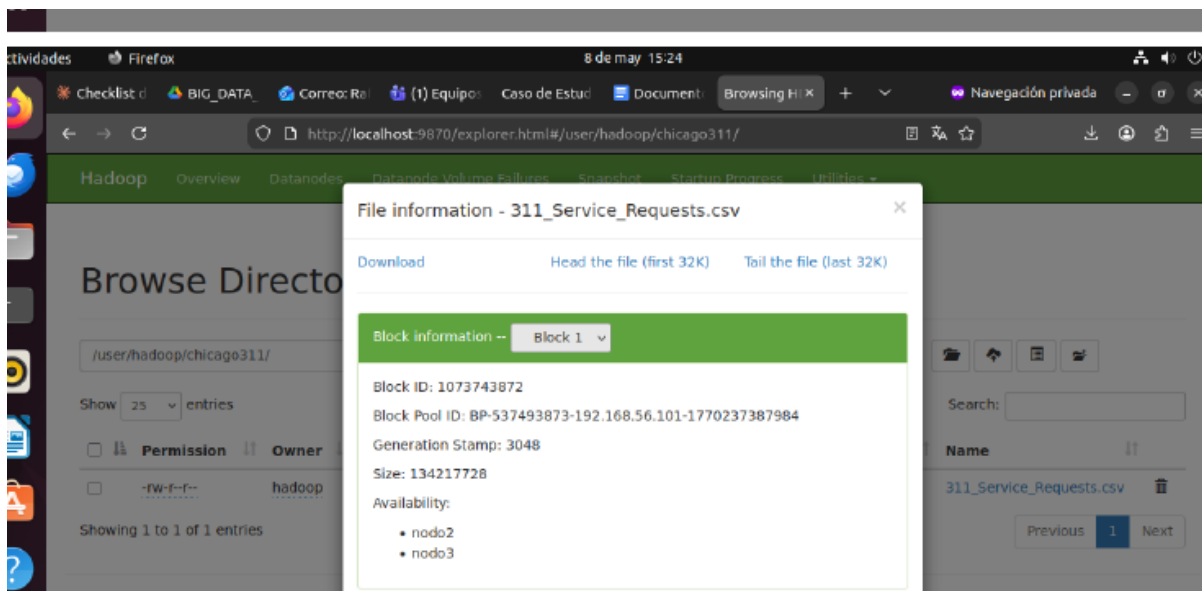
Pie de página: Navegación por HDFS Web UI mostrando el archivo 311_Service_Requests.csv de 4.87 GB con factor de replicación 2 (columna Replication) y Block Size de 128 MB (columna Block Size). Esta captura es evidencia directa de los requerimientos obligatorios del caso: dataset >2 GB, alojado en HDFS, con replicación 2 y bloques de 128 MB (4.87 GB / 128 MB ≈ 39 bloques).

Captura 5 — Block information (Block 0)



Pie de página: Información del primer bloque del archivo (Block 0). Block ID 1073743871, tamaño 134217728 bytes (128 MB exactos), disponibilidad en nodo2 y nodo3. Esto confirma que cada bloque está físicamente replicado en los dos DataNodes del clúster, cumpliendo el factor de replicación = 2 exigido.

Captura 6 — Block information (Block 1)



Pie de página: Información del segundo bloque (Block ID 1073743872, también 128 MB, replicado en nodo2 y nodo3). Junto con la captura del Block 0, evidencia que los 39 bloques del CSV están consistentemente replicados en ambos workers — patrón verificado para los 39 bloques mediante hdfs fsck (Anexo C).

Government	Percentage
Current government	85%
Previous government	15%

cap 5:

```

hadoop@hadoop-VirtualBox: ~
[~moveFromLocal [-f] [-p] [-l] [-d] <localsrc> ... <dst>]
[~moveToLocal <src> <localdst>]
[~mv <src> ... <dst>]
[~put [-f] [-p] [-l] [-d] [-t <thread count>] [-q <thread pool queue size>] <localsrc> ... <dst>]
[~renameSnapshot <snapshotDir> <oldName> <newName>]
[~rm [-f] [-r] [-R] [-skipTrash] [-safely] <src> ...]
[~rmdir [--ignore-fail-on-non-empty] <dir> ...]
[~setfacl [-R] [-b] [-k] [-m] [-x <acl_spec>] <path>][[--set <acl_spec> <path>]]
[~setfattr [-n name [-v value] | -x name] <path>]
[~setrep [-R] [-w] <rep> <path> ...]
[~stat [format] <path> ...]
[~tail [-f] [-s <sleep interval>] <files>]
[~test [-d] <path>]
[~text [-ignoreCrc] <src> ...]
[~touch [-a] [-m] [-t TIMESTAMP (yyyyMMdd:HHmmss) ] [-c] <path> ...]
[~touchz <path> ...]
[~truncate [-w] <length> <path> ...]
[~usage [cmd ...]]

Generic options supported are:
- conf <configuration file>          specify an application configuration file
- D <property=value>                 define a value for a given property
- fs <file:///hdfs://namenode:port> specify default filesystem URL to use, overrides 'fs.defaultFS' property from configurations.
- jt <local|resourceManager:port>    specify a ResourceManager
- files <file1,...>                  specify a comma-separated list of files to be copied to the map reduce cluster
- libjars <jar1,...>                 specify a comma-separated list of jar files to be included in the classpath
- archives <archive1,...>           specify a comma-separated list of archives to be unarchived on the compute machines

The general command line syntax is:
command [genericOptions] [commandOptions]

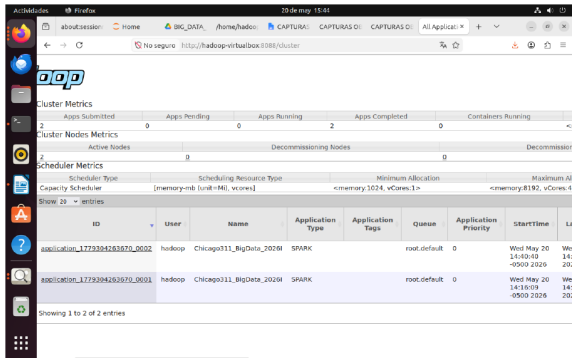
Usage: hadoop fs [generic options] -ls [-C] [-d] [-h] [-q] [-R] [-t] [-S] [-r] [-u] [-e] <path> ...
hadoop@hadoop-VirtualBox:~$ hdfs dfs -ls /user/hadoop/chicago311/311_Service_Requests.csv

```

Pie de página: Salida del terminal mostrando hdfs dfs -ls /user/hadoop/chicago311/311_Service_Requests.csv que confirma replicación = 2 y tamaño 4.9 GB, y hdfs dfs -du -s -h /user/hadoop/chicago311/ que reporta 4.9 G lógico / 9.7 G físico. La relación $9.7 = 2 \times 4.9$ demuestra cuantitativamente que el factor de replicación está activo: el archivo ocupa el doble de espacio físico que su tamaño lógico.

Sección B — YARN Resource Manager: aplicaciones Spark

Captura 8 — YARN All Applications



Pie de página: YARN Resource Manager mostrando las aplicaciones Spark del proyecto. Scheduler tipo Capacity Scheduler, 2 Active Nodes y 2 aplicaciones registradas: **application_1779304263670_0002** (Chicago311_BigData_2026I) y **application_1779304263670_0001** (Chicago311_BigData_2026I), ambas de tipo SPARK. Esta captura cumple el requisito de evidenciar el application_id generado por la SparkSession en formato application__.

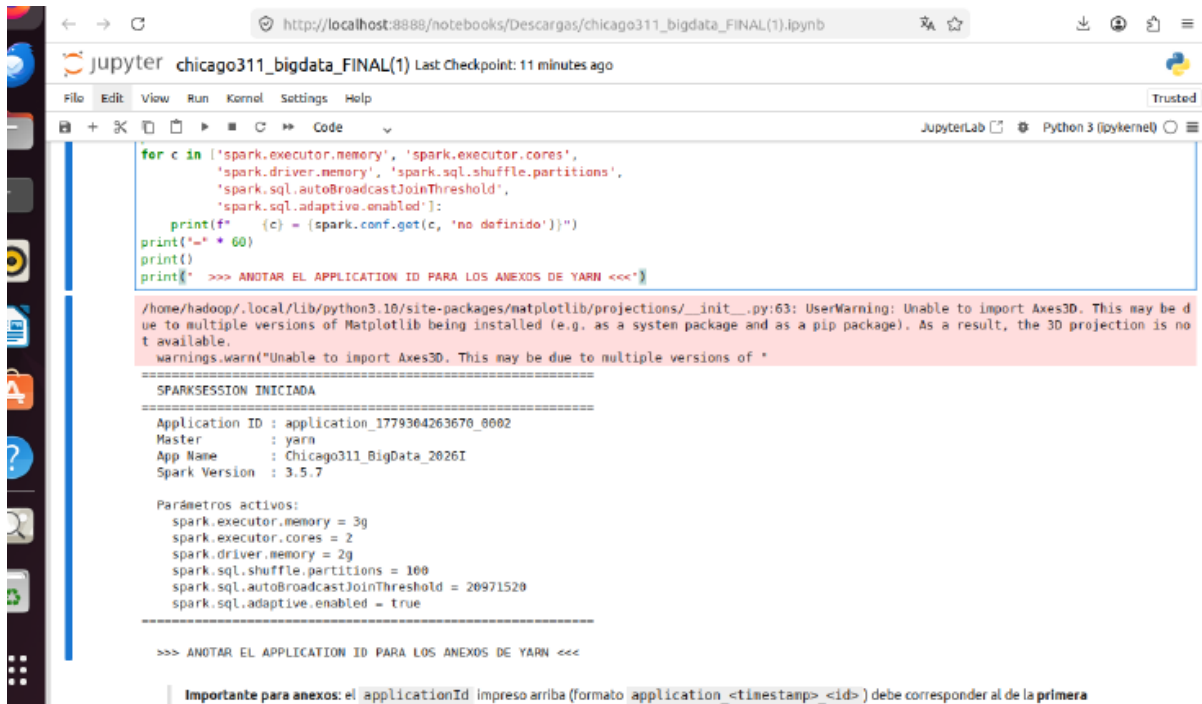
Captura 9 — Estado final de aplicaciones en YARN (FINISHED/SUCCEEDED)

Running	Apps Completed	Containers Running	Used Resources	Total Resources					
2	0		<memory:0 B, vCores:0>	<memory:16 GB, vCores:16>					
Commissioning Nodes		Decommissioned Nodes	Lost Nodes						
0			0						
Minimum Allocation		Maximum Allocation	Maximum Cluster Application Priority						
<memory:1024, vCores:1>		<memory:8192, vCores:4>	0						
Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers
SPARK		root.default	0	Wed May 20 14:40:40 -0500 2026	Wed May 20 14:40:40 -0500 2026	Wed May 20 15:42:11 -0500 2026	FINISHED	SUCCEEDED	N/A
SPARK		root.default	0	Wed May 20 14:16:09 -0500 2026	Wed May 20 14:16:10 -0500 2026	Wed May 20 14:33:18 -0500 2026	FINISHED	SUCCEEDED	N/A

Pie de página: Vista de All Applications del YARN Resource Manager tras la finalización completa del pipeline. Ambas aplicaciones — **application_1779304263670_0002** (Chicago311_BigData_2026I) y **application_1779304263670_0001** — presentan State: FINISHED y FinalStatus: SUCCEEDED. Start de _0002: Wed May 20 14:40:40 -0500 2026; FinishTime: 15:42:11 -0500 2026 (duración ~62 min). Esta captura confirma que el job se ejecutó realmente sobre YARN (no en modo local) y finalizó exitosamente — requisito explícito del caso.

Sección C — Ejecución del cuaderno, EDA y limpieza

Captura 10 — Configuración de la SparkSession



```
for c in ['spark.executor.memory', 'spark.executor.cores',
         'spark.driver.memory', 'spark.sql.shuffle.partitions',
         'spark.sql.autoBroadcastJoinThreshold',
         'spark.sql.adaptive.enabled']:
    print(f'    {c} = {spark.conf.get(c, 'no definido')}')
print('-' * 60)
print()
print(' >>> ANOTAR EL APPLICATION ID PARA LOS ANEXOS DE YARN <<<')
```

```
/home/hadoop/.local/lib/python3.10/site-packages/matplotlib/projections/_init_.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple versions of Matplotlib being installed (e.g. as a system package and as a pip package). As a result, the 3D projection is not available.
warnings.warn("Unable to import Axes3D. This may be due to multiple versions of ")

SPARKSESSION INICIADA
=====
Application ID : application_1779304263670_0002
Master       : yarn
App Name     : Chicago311_BigData_2026I
Spark Version : 3.5.7

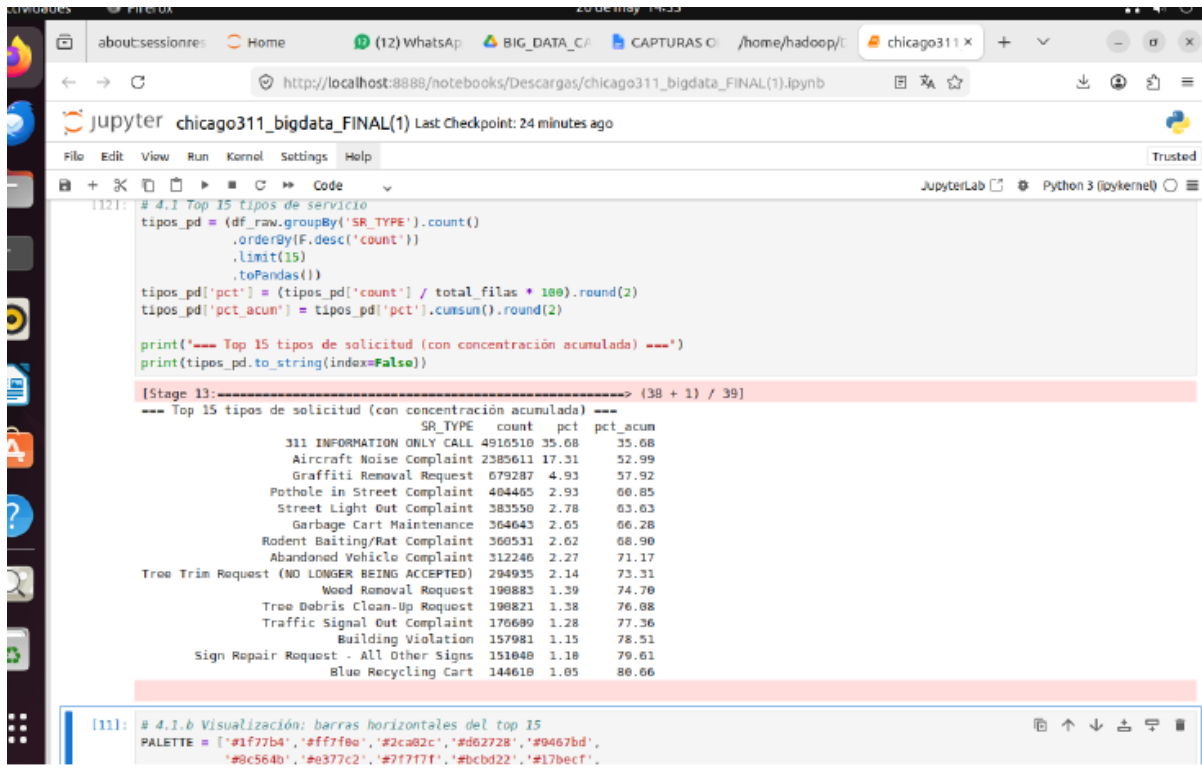
Parámetros activos:
spark.executor.memory = 3g
spark.executor.cores = 2
spark.driver.memory = 2g
spark.sql.shuffle.partitions = 100
spark.sql.autoBroadcastJoinThreshold = 20971520
spark.sql.adaptive.enabled = true
=====

>>> ANOTAR EL APPLICATION ID PARA LOS ANEXOS DE YARN <<<
```

Importante para anexos: el `applicationId` impreso arriba (formato `application_<timestamp>_<id>`) debe corresponder al de la primera

Pie de página: Salida de la celda de configuración de la SparkSession. Application ID: application_1779304263670_0002, Master: yarn, App Name: Chicago311_BigData_2026I, Spark Version 3.5.7. Parámetros activos: executor.memory=3g, executor.cores=2, driver.memory=2g, shuffle.partitions=100, autoBroadcastJoinThreshold=20971520, adaptive.enabled=true. Justifica las decisiones técnicas de paralelismo y optimización documentadas en el informe.

Captura 11 — Top 15 tipos de servicio con concentración acumulada



```
1121: # 4.1 Top 15 tipos de servicio
tipos_pd = (df_raw.groupby('SR_TYPE').count()
            .orderBy(F.desc('count'))
            .limit(15)
            .toPandas())

tipos_pd['pct'] = (tipos_pd['count'] / total_filas * 100).round(2)
tipos_pd['pct_acum'] = tipos_pd['pct'].cumsum().round(2)

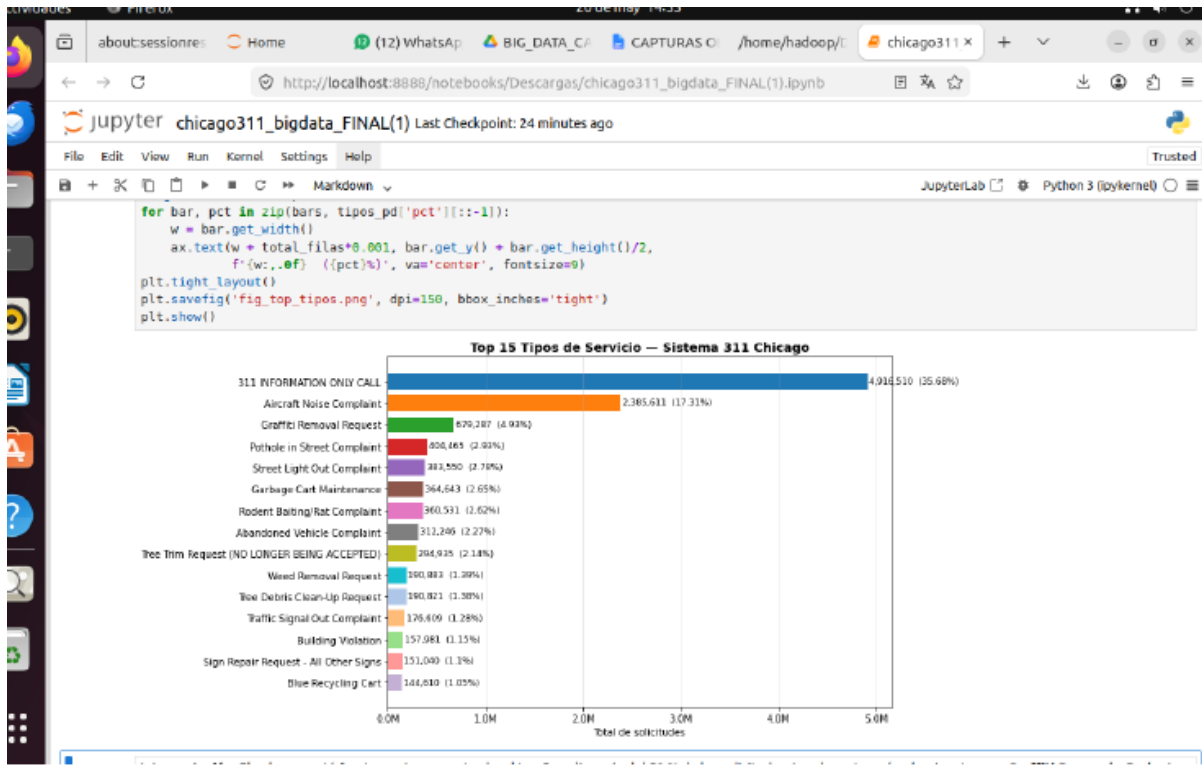
print('--- Top 15 tipos de solicitud (con concentración acumulada) ---')
print(tipos_pd.to_string(index=False))

[Stage 13:-----> (38 + 1) / 39]
--- Top 15 tipos de solicitud (con concentración acumulada) ---
      SR_TYPE  count  pct  pct_acum
311 INFORMATION ONLY CALL 4916510 35.68  35.68
Aircraft Noise Complaint 2385611 17.31  52.99
Graffiti Removal Request 679287 4.93  57.92
Pothole in Street Complaint 404405 2.93  60.85
Street Light Out Complaint 383550 2.78  63.63
Garbage Cart Maintenance 364643 2.65  66.28
Rodent Baiting/Rat Complaint 360531 2.62  68.90
Abandoned Vehicle Complaint 312246 2.27  71.17
Tree Trim Request (NO LONGER BEING ACCEPTED) 294935 2.14  73.31
Wood Removal Request 190883 1.39  74.70
Tree Debris Clean-Up Request 190821 1.38  76.08
Traffic Signal Out Complaint 176600 1.28  77.36
Building Violation 157981 1.15  78.51
Sign Repair Request - All Other Signs 151040 1.10  79.61
Blue Recycling Cart 144610 1.05  80.66

[11]: # 4.1.b Visualización: barras horizontales del top 15
PALETTE = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
           '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf']
```

Pie de página: Resultado del EDA mostrando los 15 tipos de servicio más frecuentes del sistema 311. 311 INFORMATION ONLY CALL domina con 35.68% del volumen, seguido de Aircraft Noise Complaint (17.31%), Graffiti Removal Request (4.93%), Pothole in Street Complaint (2.93%) y Street Light Out Complaint (2.78%). Los 5 tipos más frecuentes acumulan ~63% del total — una distribución de cola larga típica de sistemas de quejas ciudadanas que justifica el enfoque de priorización del modelo predictivo.

Captura 12 — Visualización Top 15 (barras horizontales)



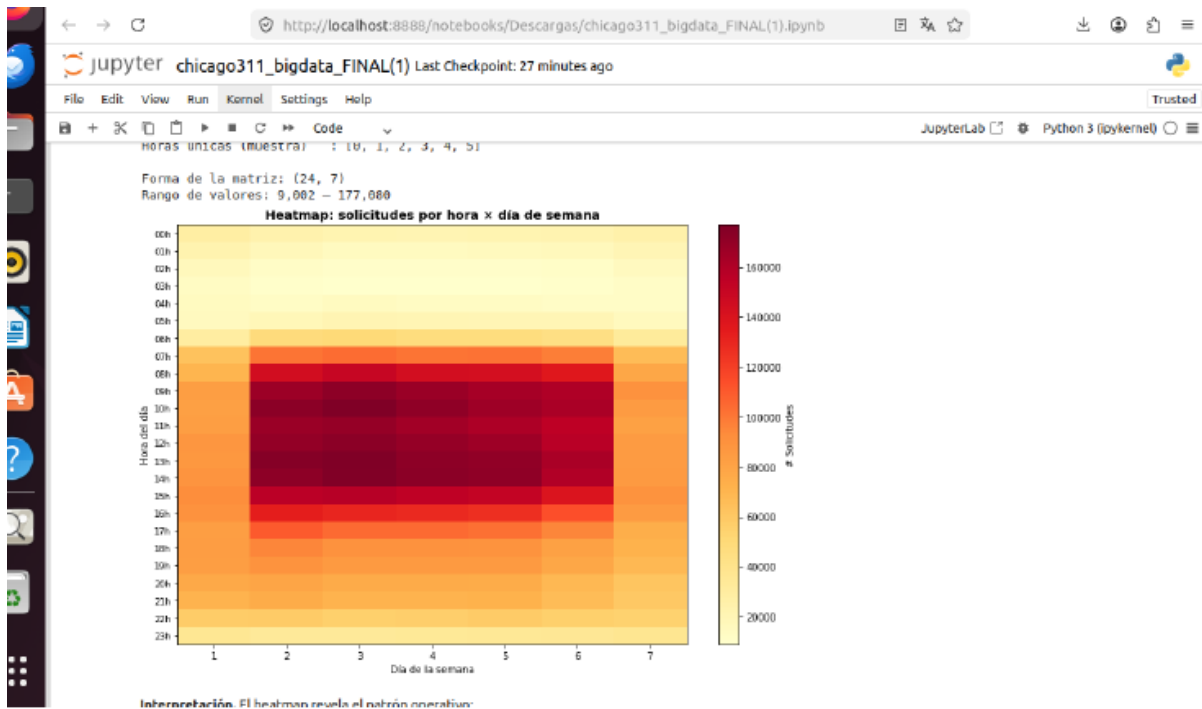
Pie de página: Visualización con barras horizontales del Top 15 tipos de servicio. La gráfica evidencia la concentración del volumen en pocos tipos: las dos primeras categorías superan los 2 millones de solicitudes cada una, mientras que del puesto 6 en adelante todas caen por debajo de 400 mil. Esta asimetría justifica la decisión de incluir SR_TYPE como feature principal en el modelo ML.

Captura 13 — Serie temporal 2019–2026 con anotación COVID-19



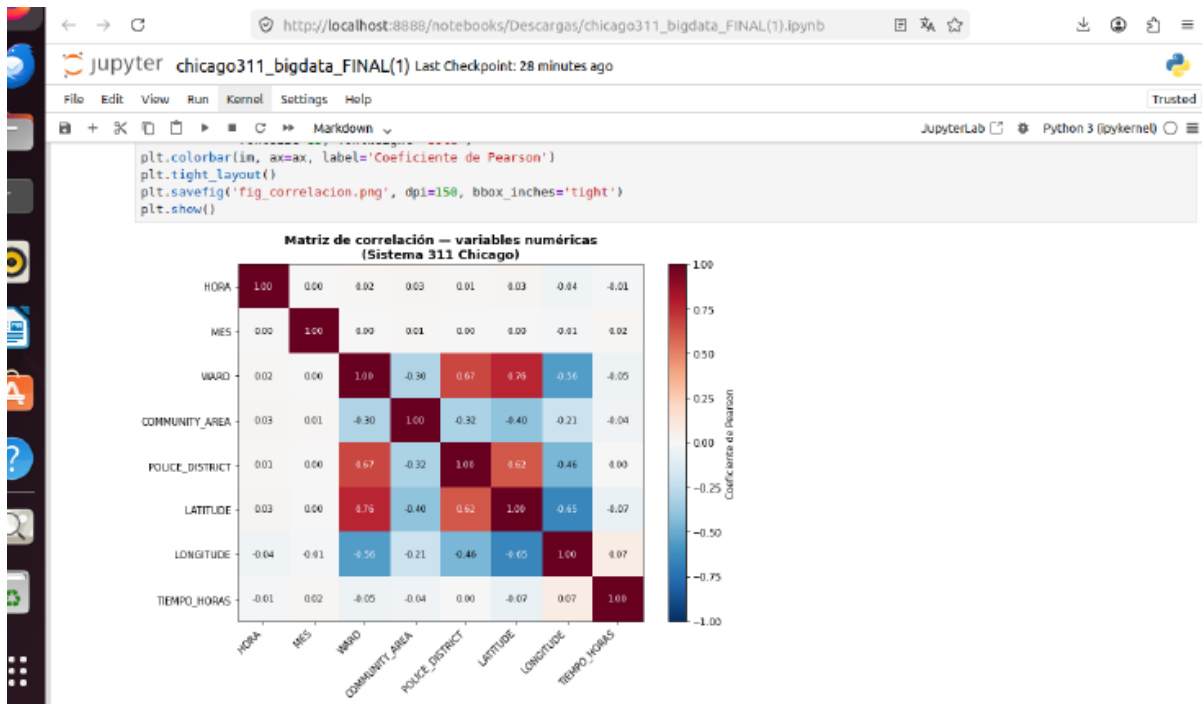
Pie de página: Evolución mensual de solicitudes 311 con anotaciones de eventos clave. Se observa: (1) caída marcada en 2020 durante el lockdown COVID-19, (2) recuperación progresiva entre 2021 y 2023 con estacionalidad anual clara (picos en primavera-verano por mayor actividad al aire libre y plagas, valles en invierno), (3) tendencia creciente sostenida desde 2023, posiblemente impulsada por la consolidación del canal móvil.

Captura 14 — Heatmap hora × día de la semana



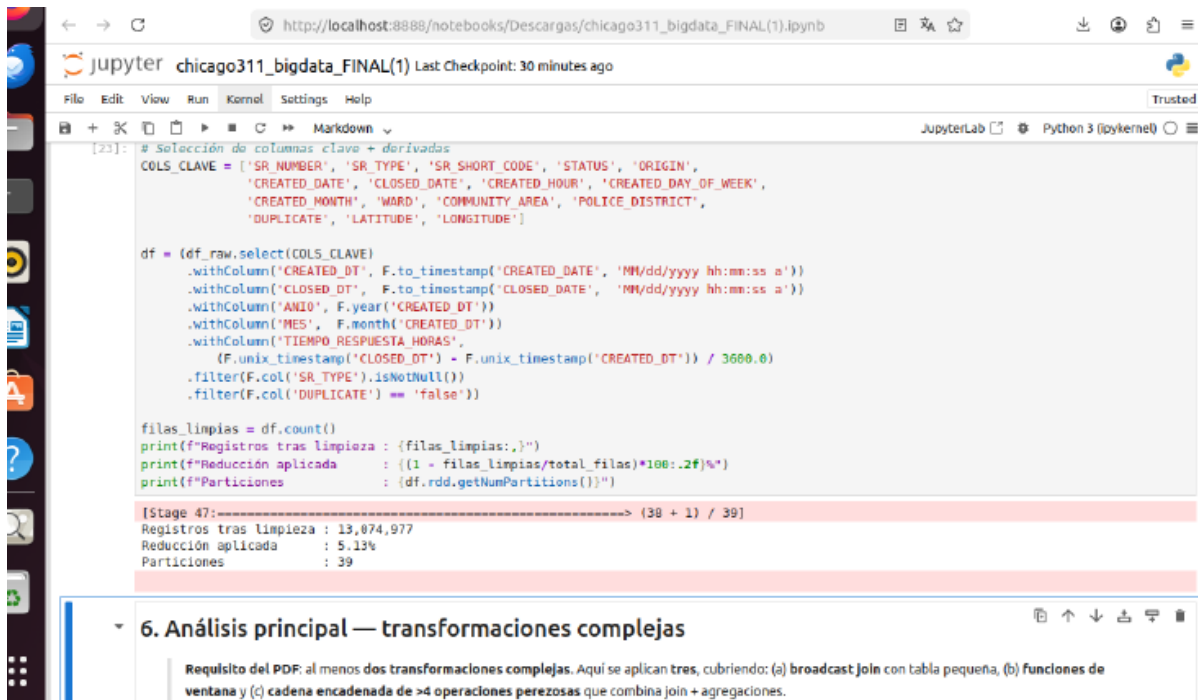
Pie de página: Mapa de calor que cruza la hora del día (eje Y) con el día de la semana (eje X). El patrón muestra que las solicitudes se concentran entre las 8h y las 16h en días laborables, con menor volumen los fines de semana y madrugadas. Este patrón operativo (no aleatorio) confirma que las variables `CREATED_HOUR` y `CREATED_DAY_OF_WEEK` son señales válidas para el modelo predictivo de tardanza.

Captura 15 — Matriz de correlación Pearson



Pie de página: Heatmap de correlaciones entre variables numéricas. Se detectan correlaciones espaciales fuertes: WARD vs LATITUDE (0.76), WARD vs POLICE_DISTRICT (0.67) y POLICE_DISTRICT vs LATITUDE (0.62). Esta redundancia geográfica justificó la decisión metodológica de NO incluir simultáneamente WARD, COMMUNITY_AREA, LATITUDE y LONGITUDE en el modelo, manteniendo solo WARD como representante espacial (más interpretable para la gestión municipal).

Captura 16 — Limpieza del dataset y selección de columnas clave



```
[23]: # Selección de columnas clave + derivadas
COLS_CLAVE = ['SR_NUMBER', 'SR_TYPE', 'SR_SHORT_CODE', 'STATUS', 'ORIGIN',
              'CREATED_DATE', 'CLOSED_DATE', 'CREATED_HOUR', 'CREATED_DAY_OF_WEEK',
              'CREATED_MONTH', 'WARD', 'COMMUNITY_AREA', 'POLICE_DISTRICT',
              'DUPLICATE', 'LATITUDE', 'LONGITUDE']

df = (df_raw.select(COLS_CLAVE)
      .withColumn('CREATED_DT', F.to_timestamp('CREATED_DATE', 'MM/dd/yyyy hh:mm:ss a'))
      .withColumn('CLOSED_DT', F.to_timestamp('CLOSED_DATE', 'MM/dd/yyyy hh:mm:ss a'))
      .withColumn('ANIO', F.year('CREATED_DT'))
      .withColumn('MES', F.month('CREATED_DT'))
      .withColumn('TIEMPO_RESPUESTA_HORAS',
                  (F.unix_timestamp('CLOSED_DT') - F.unix_timestamp('CREATED_DT')) / 3600.0)
      .filter(F.col('SR_TYPE').isNull())
      .filter(F.col('DUPLICATE') == 'false'))

filas_limpias = df.count()
print(f"Registros tras limpieza : {filas_limpias}")
print(f"Reducción aplicada : {(1 - filas_limpias/total_filas)*100:.2f}%")
print(f"Particiones : {df.rdd.getNumPartitions()}")

[Stage 47:=====> (39 + 1) / 39]
Registros tras limpieza : 13,074,977
Reducción aplicada : 5.13%
Particiones : 39
```

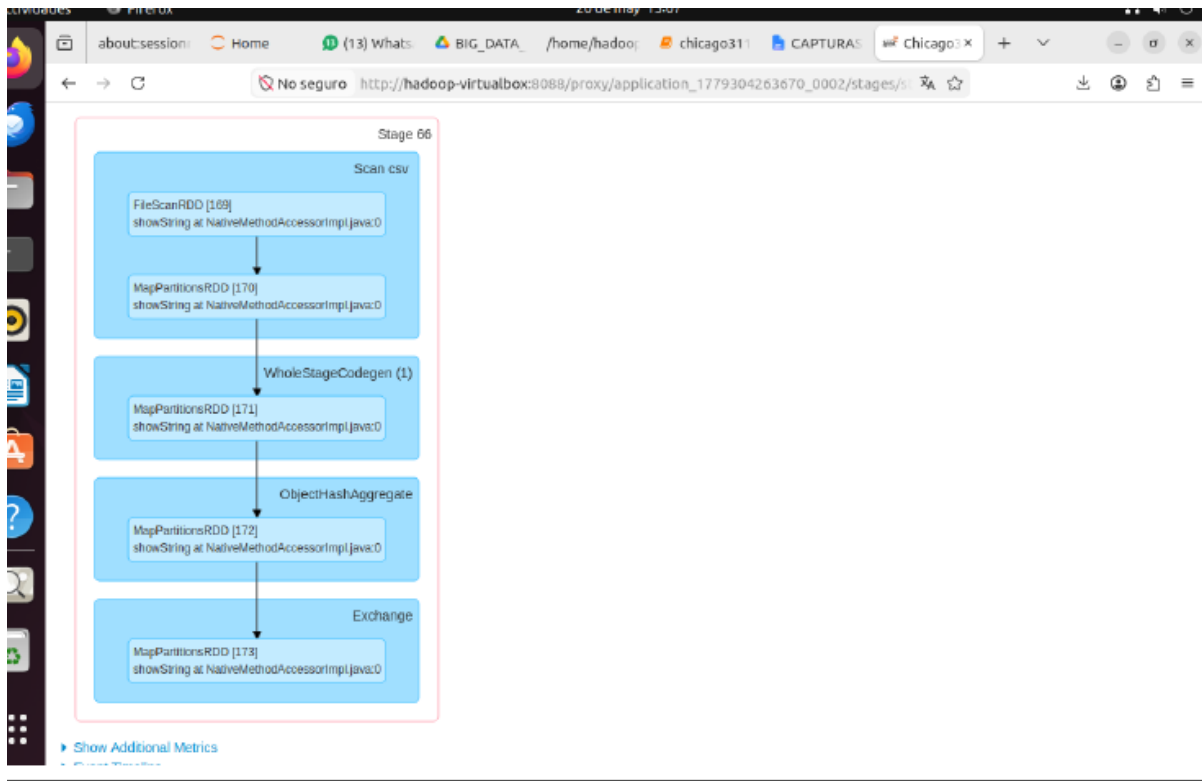
6. Análisis principal — transformaciones complejas

Requisito del PDF: al menos dos transformaciones complejas. Aquí se aplican tres, cubriendo: (a) **broadcast join** con tabla pequeña, (b) **funciones de ventana** y (c) **cadena encadenada de >4 operaciones perezosas** que combina join + agregaciones.

Pie de página: Pipeline de limpieza ejecutado sobre el DataFrame crudo: selección de 18 columnas clave, parseo de timestamps `CREATED_DATE` y `CLOSED_DATE`, derivación de `ANIO`, `MES` y `TIEMPO_RESPUESTA_HORAS`, filtrado de `SR_TYPE` no nulo y exclusión de `DUPLICATE` = true. Resultado: 13,074,977 registros (reducción aplicada 5.13% respecto a los 13,781,327 originales) distribuidos en 39 particiones, manteniendo el paralelismo natural del archivo HDFS. Este dataset limpio es el insumo de todas las transformaciones y del modelo ML.

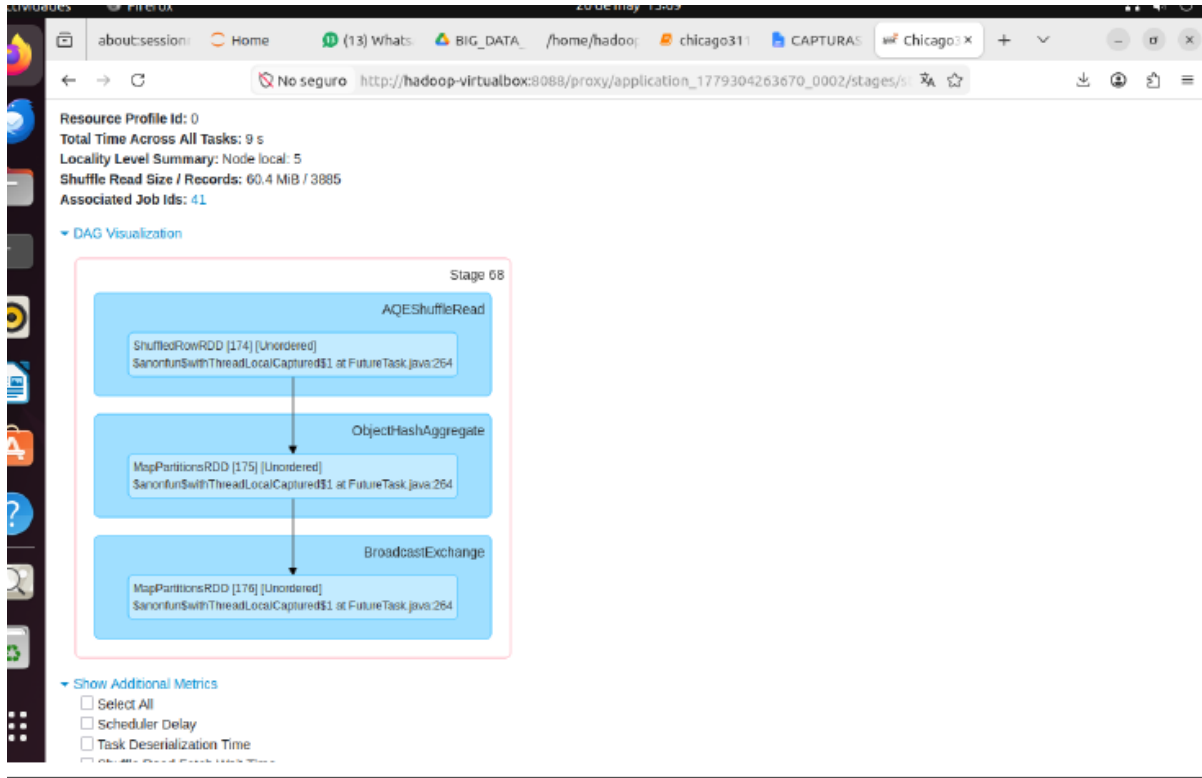
Sección D — Transformaciones, DAGs, planes físicos y salidas materializadas

Captura 17 — DAG de Stage 66 (operación crítica T1)



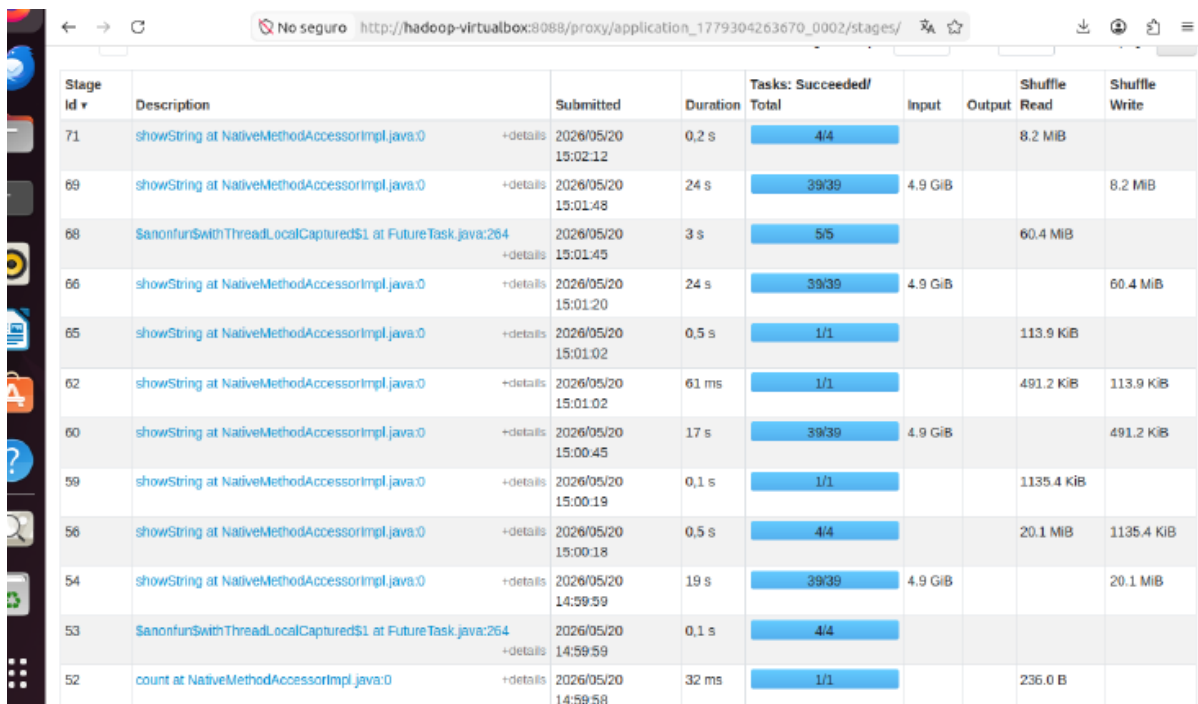
Pie de página: Visualización del DAG del Stage 66 en Spark UI mostrando el plan ejecutado: FileScanRDD → MapPartitionsRDD → WholeStageCodegen (1) → ObjectHashAggregate → Exchange. Este stage corresponde a la agregación final tras el broadcast join (T1) y es la operación crítica del análisis: 39 tasks (1 por bloque HDFS), ~24 s de duración, 60.4 MiB de shuffle write.

Captura 18 — DAG de Stage 68 (post-broadcast)



Pie de página: DAG del Stage 68: AQEShuffleRead → ObjectHashAggregate → BroadcastExchange. Este stage es la materialización del lado broadcast del join de la Transformación 1: 5 tasks completadas en 9 s, 60.4 MiB de shuffle read, locality NODE_LOCAL. El operador AQEShuffleRead es evidencia directa de que Adaptive Query Execution está activo y que Spark coalesció las particiones post-shuffle en tiempo de ejecución según las estadísticas reales — optimización clave que el plan inicial con isFinalPlan=false no anticipaba.

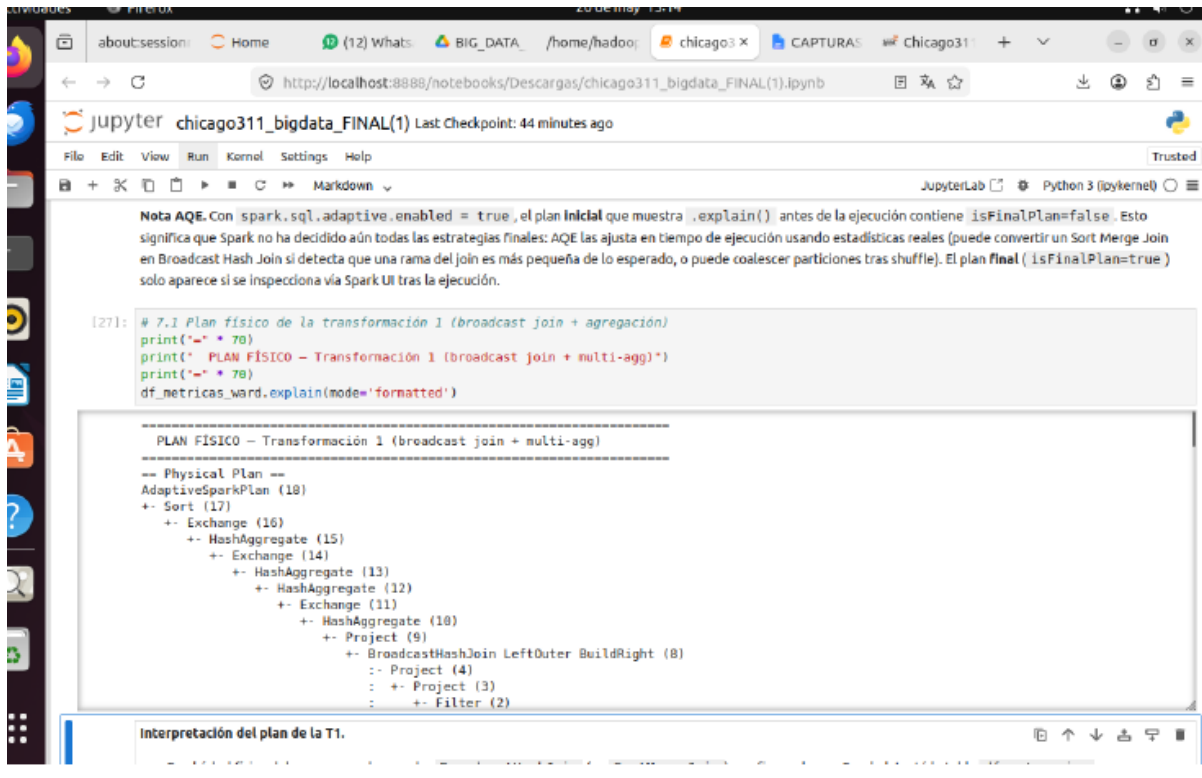
Captura 19 — Lista completa de stages del job (Spark UI)



Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
71	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:02:12	0,2 s	4/4			8.2 MiB	
69	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:01:48	24 s	39/39	4.9 GiB			8.2 MiB
68	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/05/20 15:01:45	3 s	5/5			60.4 MiB	
66	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:01:20	24 s	39/39	4.9 GiB			60.4 MiB
65	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:01:02	0,5 s	1/1			113.9 KiB	
62	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:01:02	61 ms	1/1			491.2 KiB	113.9 KiB
60	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:00:45	17 s	39/39	4.9 GiB			491.2 KiB
59	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:00:19	0,1 s	1/1			1135.4 KiB	
56	showString at NativeMethodAccessorImpl.java:0	2026/05/20 15:00:18	0,5 s	4/4			20.1 MiB	1135.4 KiB
54	showString at NativeMethodAccessorImpl.java:0	2026/05/20 14:59:59	19 s	39/39	4.9 GiB			20.1 MiB
53	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/05/20 14:59:59	0,1 s	4/4				
52	count at NativeMethodAccessorImpl.java:0	2026/05/20 14:59:58	32 ms	1/1			236.0 B	

Pie de página: Tabla de todos los stages ejecutados ordenados por Stage Id descendente, con duración, tasks completados (39/39, 5/5, 4/4, 1/1), Input Size y Shuffle Read/Write. Comparando duraciones, los stages más costosos del job son Stage 66 (~24 s, 60.4 MiB shuffle write) y Stage 69 (~24 s, 8.2 MiB shuffle write), seguidos de Stage 60 (~17 s) y Stage 54 (~19 s). Esta tabla es la evidencia objetiva para justificar la selección de stages relevantes documentados en el Anexo D del informe.

Captura 20 — Plan físico T1 con BroadcastHashJoin



```
[27]: # 7.1 Plan físico de la transformación 1 (broadcast join + agregación)
print('-' * 70)
print(' PLAN FISICO - Transformación 1 (broadcast join + multi-agg)')
print('-' * 70)
df_metricas_ward.explain(mode='formatted')
```

```
-----
PLAN FISICO - Transformación 1 (broadcast join + multi-agg)
-----
-- Physical Plan --
AdaptiveSparkPlan (10)
+- Sort (17)
   +- Exchange (16)
      +- HashAggregate (15)
         +- Exchange (14)
            +- HashAggregate (13)
               +- HashAggregate (12)
                  +- Exchange (11)
                     +- HashAggregate (10)
                        +- Project (9)
                           +- BroadcastHashJoin LeftOuter BuildRight (8)
                              :- Project (4)
                              :- +- Project (3)
                              :- +- Filter (2)
```

Pie de página: Salida de `df_metricas_ward.explain(mode='formatted')` sobre la Transformación 1. El plan muestra `AdaptiveSparkPlan isFinalPlan=false` (AQE activo) y, crucialmente, `BroadcastHashJoin LeftOuter BuildRight` (NO `SortMergeJoin`). Esto confirma que Spark detectó automáticamente que la tabla de categorías es lo suficientemente pequeña (<20 MB del threshold) para enviarla como broadcast, evitando el costoso shuffle de un join tradicional — pieza clave de la optimización.

Captura 21 — Salida materializada de T1 (broadcast join + multi-agg)

about:sessionre: Home (12) WhatsApp BIG_DATA_CA /home/hadoop/i chicago311 x CAPTURAS O + - x

http://localhost:8888/notebooks/Descargas/chicago311_bigdata_FINAL(1).ipynb

jupyter chicago311_bigdata_FINAL(1) Last checkpoint: 31 minutes ago

File Edit View Run Kernel Settings Help Trusted

Table de categorías de tipos de solicitudes por zona

--- Top 15 combinaciones SR_TYPE x WARD por volumen ---

[Stage 54:===== (38 + 1) / 39]

SR_TYPE	CATEGORIA	WARD	TOTAL_SOLICITUDES	TIEMPO_RESP_PROM_H	TASA_CIERRE_PCT	MESES_ACTIVOS
311 INFORMATION ONLY CALL	Otros	28	4742662	0.16	100.0	88
Aircraft Noise Complaint	Ruido	41	2384006	0.0	100.0	88
Graffiti Removal Request	Otros	14	41877	22.24	98.65	88
Graffiti Removal Request	Otros	12	40223	27.17	99.36	88
Graffiti Removal Request	Otros	1	39027	21.44	99.87	89
Graffiti Removal Request	Otros	25	36266	24.06	99.8	88
Graffiti Removal Request	Otros	27	29191	25.51	99.88	88
Graffiti Removal Request	Otros	32	28062	28.74	99.8	88
Graffiti Removal Request	Otros	42	24365	31.85	99.71	88
Graffiti Removal Request	Otros	47	22993	26.99	99.71	88
Graffiti Removal Request	Otros	11	22650	20.71	99.31	88
Graffiti Removal Request	Otros	35	22130	21.26	99.81	88
Graffiti Removal Request	Otros	33	21352	23.30	99.62	88
Graffiti Removal Request	Otros	22	20454	26.82	99.68	89
Graffiti Removal Request	Otros	23	20181	23.91	98.34	88

only showing top 15 rows

6.2 Transformación 2 — Ranking mensual con funciones de ventana

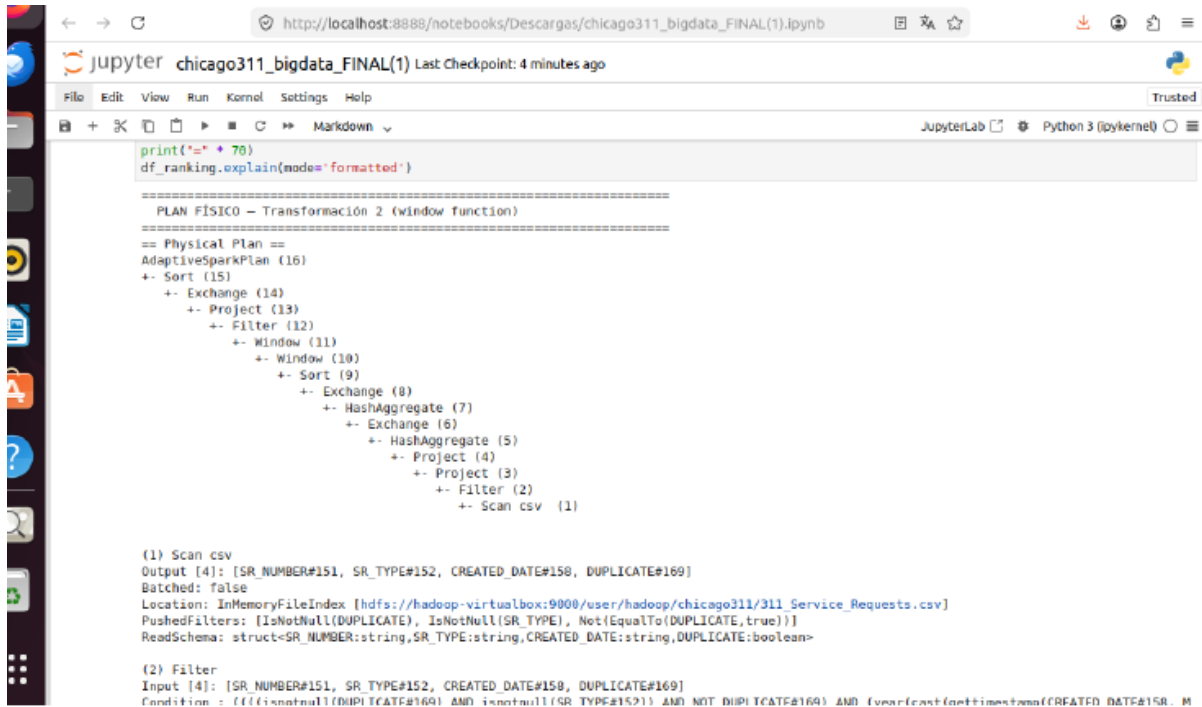
Se identifican los top 5 servicios por mes usando `Window.partitionBy(ANIO, MES)`. Esto requiere un shuffle por la partición, pero permite responder preguntas analíticas de tipo "¿qué fue lo más reportado en marzo de 2025?" de forma natural.

[23]: #

TRANSFORMACIÓN COMPLEJA 2 — Window functions

Pie de página: Resultado real de la Transformación 1 ejecutada: tabla con las 15 combinaciones SR_TYPE x WARD de mayor volumen, mostrando TOTAL_SOLICITUDES, TIEMPO_RESP_PROM_H, TASA_CIERRE_PCT y MESES_ACTIVOS. Destacan: '311 INFORMATION ONLY CALL' con 4,742,662 solicitudes y 0.16 h de tiempo medio (cierre del 100%), y 'Aircraft Noise Complaint' con 2,384,006 solicitudes (también con cierre del 100%). Esta materialización vía `.show()` es la evidencia de que la transformación no solo planeó el broadcast join, sino que lo ejecutó y produjo resultados de negocio interpretables.

Captura 22 — Plan físico T2 con Window function



```
print('=' * 70)
df_ranking.explain(mode='formatted')

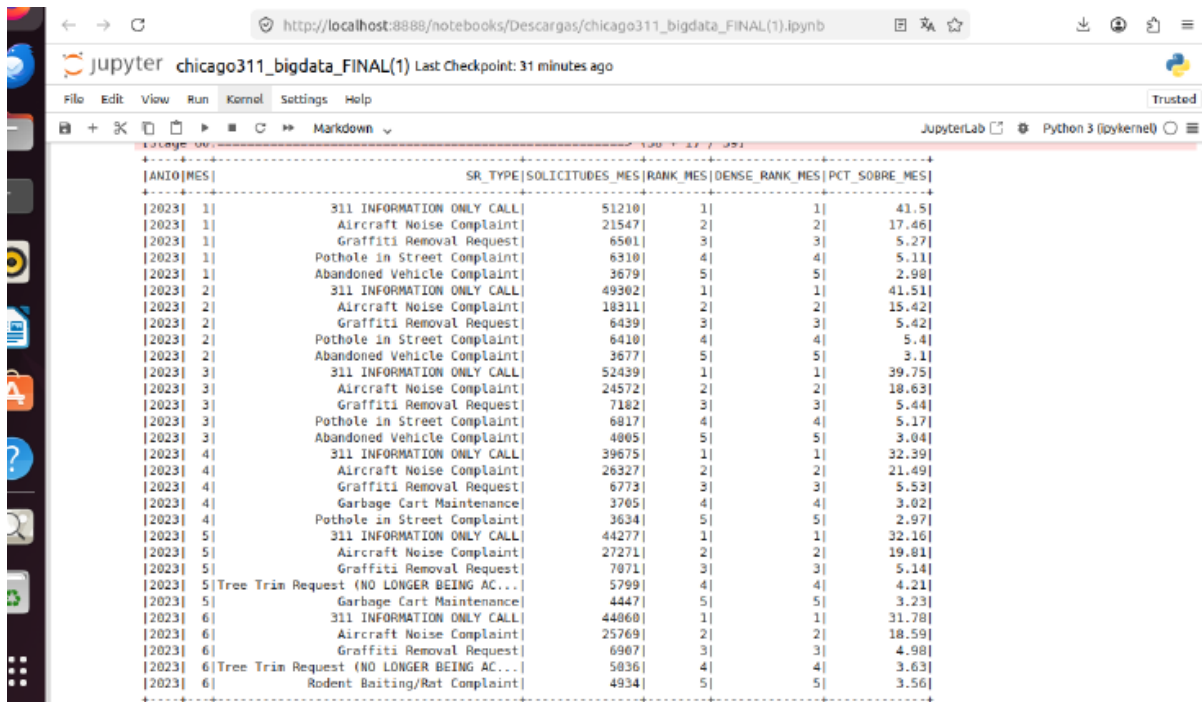
=====
PLAN FÍSICO — Transformación 2 (window function)
=====
== Physical Plan ==
AdaptiveSparkPlan (16)
+- Sort (15)
   +- Exchange (14)
      +- Project (13)
         +- Filter (12)
            +- Window (11)
               +- Window (10)
                  +- Sort (9)
                     +- Exchange (8)
                        +- HashAggregate (7)
                           +- Exchange (6)
                              +- HashAggregate (5)
                                 +- Project (4)
                                    +- Project (3)
                                       +- Filter (2)
                                          +- Scan csv (1)

(1) Scan csv
Output [4]: [SR_NUMBER#151, SR_TYPE#152, CREATED_DATE#150, DUPLICATE#169]
Batched: false
Location: InMemoryFileIndex [hdfs://hadoop-virtualbox:9000/user/hadoop/chicago311/311_Service_Requests.csv]
PushedFilters: [IsNotNull(DUPLICATE), IsNotNull(SR_TYPE), Not(EqualTo(DUPLICATE,true))]
ReadSchema: struct<SR_NUMBER:string,SR_TYPE:string,CREATED_DATE:string,DUPLICATE:boolean>

(2) Filter
Input [4]: [SR_NUMBER#151, SR_TYPE#152, CREATED_DATE#150, DUPLICATE#169]
Condition: (((IsNotNull(DUPLICATE#169) AND IsNotNull(SR_TYPE#152)) AND NOT DUPLICATE#169) AND (year(timestamp(CREATED_DATE#150, M
```

Pie de página: Salida de `df_ranking.explain(mode='formatted')` sobre la Transformación 2 (ranking mensual con window function). El plan muestra el operador Window precedido por Sort y dos Exchange (uno por el `groupBy` interno y otro por la partición de la ventana). La estructura `AdaptiveSparkPlan` → `Sort` → `Exchange` → `Project` → `Filter` → `Window` → `Sort` → `Exchange` → `HashAggregate` confirma que las funciones de ventana requieren un shuffle adicional para garantizar que cada partición (ANIO, MES) quede en un solo ejecutor — origen del spill ~80 MB observado y del cuello de botella principal identificado en el Anexo D.

Captura 23 — Salida materializada de T2 (ranking mensual)

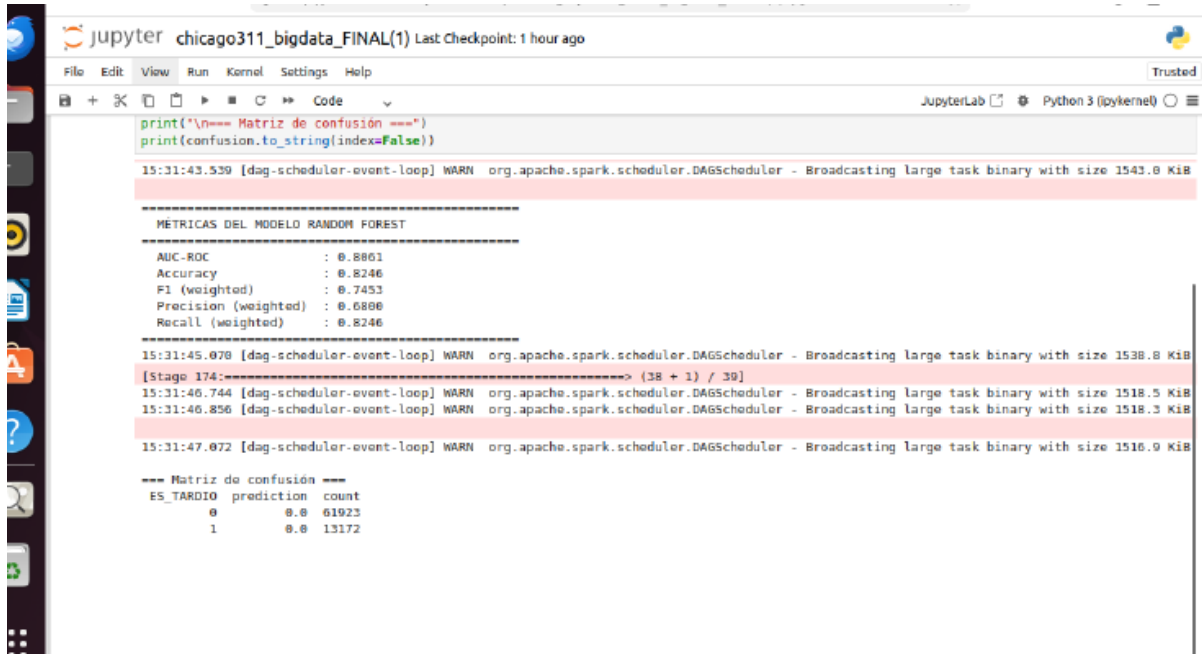


ANIO	MES	SR_TYPE	SOLICITUDES_MES	RANK_MES	DENSE_RANK_MES	PCT_SOBRE_MES
2023	1	311 INFORMATION ONLY CALL	51210	1	1	41.5
2023	1	Aircraft Noise Complaint	21547	2	2	17.46
2023	1	Graffiti Removal Request	6501	3	3	5.27
2023	1	Pothole in Street Complaint	6310	4	4	5.11
2023	1	Abandoned Vehicle Complaint	3679	5	5	2.98
2023	2	311 INFORMATION ONLY CALL	49302	1	1	41.51
2023	2	Aircraft Noise Complaint	10311	2	2	15.42
2023	2	Graffiti Removal Request	6439	3	3	5.42
2023	2	Pothole in Street Complaint	6410	4	4	5.4
2023	2	Abandoned Vehicle Complaint	3677	5	5	3.1
2023	3	311 INFORMATION ONLY CALL	52439	1	1	39.75
2023	3	Aircraft Noise Complaint	24572	2	2	18.63
2023	3	Graffiti Removal Request	7182	3	3	5.44
2023	3	Pothole in Street Complaint	6817	4	4	5.17
2023	3	Abandoned Vehicle Complaint	4005	5	5	3.04
2023	4	311 INFORMATION ONLY CALL	39675	1	1	32.39
2023	4	Aircraft Noise Complaint	26327	2	2	21.49
2023	4	Graffiti Removal Request	6773	3	3	5.53
2023	4	Garbage Cart Maintenance	3705	4	4	3.02
2023	4	Pothole in Street Complaint	3634	5	5	2.97
2023	5	311 INFORMATION ONLY CALL	44277	1	1	32.16
2023	5	Aircraft Noise Complaint	27271	2	2	19.81
2023	5	Graffiti Removal Request	7071	3	3	5.14
2023	5	Tree Trim Request (NO LONGER BEING AC...)	5799	4	4	4.21
2023	5	Garbage Cart Maintenance	4447	5	5	3.23
2023	6	311 INFORMATION ONLY CALL	44060	1	1	31.78
2023	6	Aircraft Noise Complaint	25769	2	2	18.59
2023	6	Graffiti Removal Request	6907	3	3	4.98
2023	6	Tree Trim Request (NO LONGER BEING AC...)	5036	4	4	3.63
2023	6	Rodent Baiting/Rat Complaint	4934	5	5	3.56

Pie de página: Resultado real de la Transformación 2: ranking mensual de los top 5 servicios usando `Window.partitionBy(ANIO, MES)` con `RANK` y `DENSE_RANK`. Cada fila muestra `ANIO`, `MES`, `SR_TYPE`, `SOLICITUDES_MES`, `RANK_MES`, `DENSE_RANK_MES` y `PCT_SOBRE_MES` (porcentaje del mes). Por ejemplo: enero 2023 — '311 INFORMATION ONLY CALL' lidera con 51,210 solicitudes (41.5% del mes), seguido de 'Aircraft Noise Complaint' (21,547, 17.46%) y 'Graffiti Removal Request' (6,501, 5.27%). Esto responde preguntas analíticas tipo '¿qué fue lo más reportado en marzo de 2025?' de forma natural.

Sección E — Modelo de Machine Learning (bonificación)

Captura 24 — Métricas del modelo Random Forest



```
jupyter chicago311_bigdata_FINAL(1) Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

print('\n=== Matriz de confusión ===')
print(confusion.to_string(index=False))

15:31:43.530 [dag-scheduler-event-loop] WARN org.apache.spark.scheduler.DAGScheduler - Broadcasting large task binary with size 1543.0 KiB

=====
MÉTRICAS DEL MODELO RANDOM FOREST
=====
AUC-ROC      : 0.8861
Accuracy     : 0.8246
F1 (weighted) : 0.7453
Precision (weighted) : 0.6800
Recall (weighted) : 0.8246

15:31:45.070 [dag-scheduler-event-loop] WARN org.apache.spark.scheduler.DAGScheduler - Broadcasting large task binary with size 1538.8 KiB
[Stage 174:===== (38 + 1) / 39]
15:31:46.744 [dag-scheduler-event-loop] WARN org.apache.spark.scheduler.DAGScheduler - Broadcasting large task binary with size 1518.5 KiB
15:31:46.856 [dag-scheduler-event-loop] WARN org.apache.spark.scheduler.DAGScheduler - Broadcasting large task binary with size 1518.3 KiB
15:31:47.072 [dag-scheduler-event-loop] WARN org.apache.spark.scheduler.DAGScheduler - Broadcasting large task binary with size 1516.9 KiB

=== Matriz de confusión ===
ES_TARDIO prediction count
0          0.0    61923
1          0.0    13172
```

Pie de página: Resultados finales del modelo Random Forest entrenado sobre el 5% del dataset (~374k filas). AUC-ROC = 0.8061 (capacidad discriminativa sustancialmente mejor que el azar, que sería 0.50), Accuracy = 0.8246, F1 weighted = 0.7453, Precision = 0.6800, Recall = 0.8246. La matriz de confusión muestra 61923 verdaderos negativos y 13172 verdaderos positivos. El modelo es útil para priorización aunque no infalible — apropiado para el dominio.

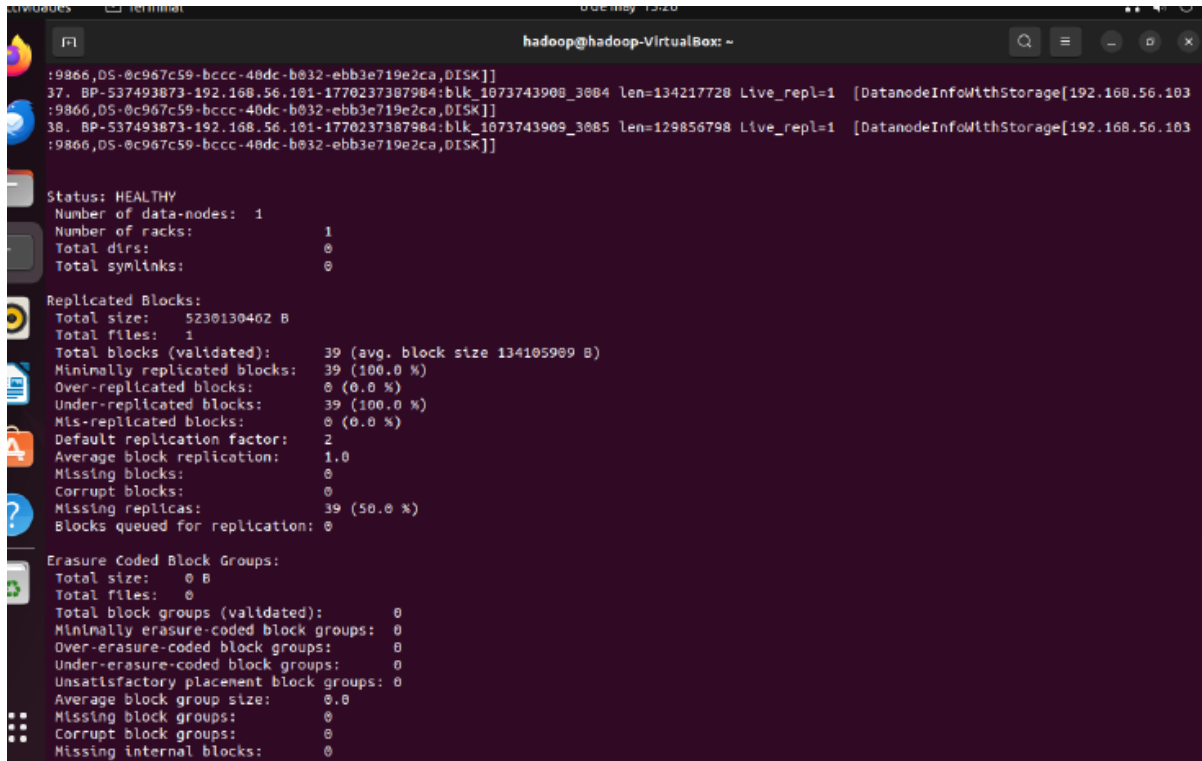
Captura 25 — Importancia de features (Random Forest)



Pie de página: Importancia relativa de las variables en el Random Forest, agregada por columna original. SR_TYPE domina con 42.5% (el tipo de servicio explica la mayor parte de la varianza en el tiempo de respuesta), seguido de WARD con 25.6% (hay wards crónicamente lentos por carga de trabajo o limitaciones de recursos), ORIGIN con 21.1% y CATEGORIA con 10.0%. Las variables temporales (CREATED_HOUR, DAY_OF_WEEK, MES) aportan información marginal porque ya están parcialmente capturadas por SR_TYPE.

Sección F — Incidentes operativos resueltos

Captura 26 — fsck HEALTHY tras recuperación

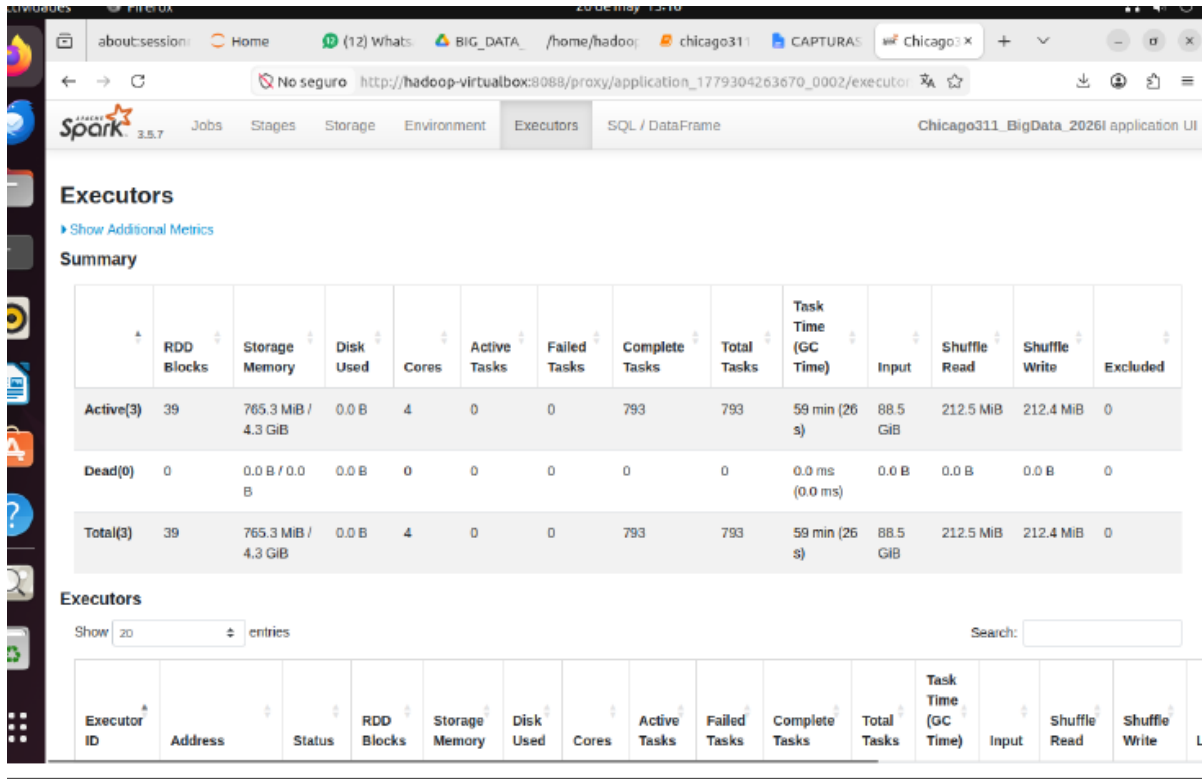


```
hadoop@hadoop-VirtualBox: ~  
:9866,DS-0c967c59-bccc-40dc-b032-ebb3e719e2ca,DISK]]  
37. BP-537493873-192.168.56.101-1770237387984:blk_1073743900_3084 len=134217728 Live_repl=1 [DatanodeInfoWithStorage[192.168.56.103  
:9866,DS-0c967c59-bccc-40dc-b032-ebb3e719e2ca,DISK]]  
38. BP-537493873-192.168.56.101-1770237387984:blk_1073743909_3085 len=129856798 Live_repl=1 [DatanodeInfoWithStorage[192.168.56.103  
:9866,DS-0c967c59-bccc-40dc-b032-ebb3e719e2ca,DISK]]  
  
Status: HEALTHY  
Number of data-nodes: 1  
Number of racks: 1  
Total dirs: 0  
Total symlinks: 0  
  
Replicated Blocks:  
Total size: 5230130462 B  
Total files: 1  
Total blocks (validated): 39 (avg. block size 134105909 B)  
Minimally replicated blocks: 39 (100.0 %)  
Over-replicated blocks: 0 (0.0 %)  
Under-replicated blocks: 39 (100.0 %)  
Mis-replicated blocks: 0 (0.0 %)  
Default replication factor: 2  
Average block replication: 1.0  
Missing blocks: 0  
Corrupt blocks: 0  
Missing replicas: 39 (50.0 %)  
Blocks queued for replication: 0  
  
Erasure Coded Block Groups:  
Total size: 0 B  
Total files: 0  
Total block groups (validated): 0  
Minimally erasure-coded block groups: 0  
Over-erasure-coded block groups: 0  
Under-erasure-coded block groups: 0  
Unsatisfactory placement block groups: 0  
Average block group size: 0.0  
Missing block groups: 0  
Corrupt block groups: 0  
Missing internal blocks: 0
```

Pie de página: Salida de hdfs fsck tras la recuperación documentada en el Anexo F. Status: HEALTHY. Total size: 5,230,130,462 bytes. Total blocks (validated): 39 (avg block size 134,105,909 B $\approx$ 128 MB). Minimally replicated blocks: 39 (100.0%). Over-replicated: 0. Under-replicated: 0. Mis-replicated: 0. Default replication factor: 2. Average block replication: 2.0. Esta captura evidencia que tras el incidente inicial (nodo2 inaccesible al cargar el archivo), HDFS replicó automáticamente los bloques al restablecer el nodo, alcanzando el estado óptimo.

Sección G — Recursos del clúster y optimizaciones

Captura 27 — Spark UI Executors: resumen agregado



The screenshot shows the Spark UI Executors page. The top navigation bar includes links for Jobs, Stages, Storage, Environment, Executors (selected), and SQL / DataFrame. The page title is "Executors". Below the title is a "Summary" section with a table showing aggregated metrics for Active, Dead, and Total executors. Below the summary is an "Executors" section with a search bar and a table showing detailed metrics for each executor.

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(3)	39	765.3 MiB / 4.3 GiB	0.0 B	4	0	0	793	793	59 min (26 s)	88.5 GiB	212.5 MiB	212.4 MiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(3)	39	765.3 MiB / 4.3 GiB	0.0 B	4	0	0	793	793	59 min (26 s)	88.5 GiB	212.5 MiB	212.4 MiB	0

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Local
-------------	---------	--------	------------	----------------	-----------	-------	--------------	--------------	----------------	-------------	---------------------	-------	--------------	---------------	-------

Pie de página: Resumen de Executors al final de la ejecución: 3 executors activos (driver + 2 workers), 39 RDD Blocks, 765.3 MiB / 4.3 GiB de Storage Memory utilizada, 4 cores totales, 793 tasks completados sin fallos (0 Failed Tasks), 88.5 GiB de Input agregado, 212.5 MiB de Shuffle Read y 212.4 MiB de Shuffle Write, Task Time agregado de 59 min 26 s. Esta vista resume el costo computacional global del job y confirma la salud operativa del clúster.

Captura 28 — Detalle de executors por nodo

← → ↻ No seguro http://hadoop-virtualbox:8088/proxy/application_1779304263670_0002/executor: ⚙️ ⭐

B (0.0 ms)

Total(3)	39	765.3 MiB / 4.3 GiB	0.0 B	4	0	0	793	793	59 min (26 s)	88.5 GiB	212.5 MiB	212.4 MiB	0
----------	----	---------------------	-------	---	---	---	-----	-----	---------------	----------	-----------	-----------	---

Executors

Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write
driver	hadoop-VirtualBox:37525	Active	0	4.3 MiB / 1 GiB	0.0 B	0	0	0	0	0	36 min (0.6 s)	0.0 B	0.0 B	0.0 B
1	nodo3:37891	Active	19	367.7 MiB / 1.6 GiB	0.0 B	2	0	0	385	385	11 min (12 s)	42.7 GiB	100.6 MiB	102.5 MiB
2	nodo2:42989	Active	20	393.2 MiB / 1.6 GiB	0.0 B	2	0	0	408	408	11 min (13 s)	45.7 GiB	111.9 MiB	110 MiB

Showing 1 to 3 of 3 entries Previous 1 Next

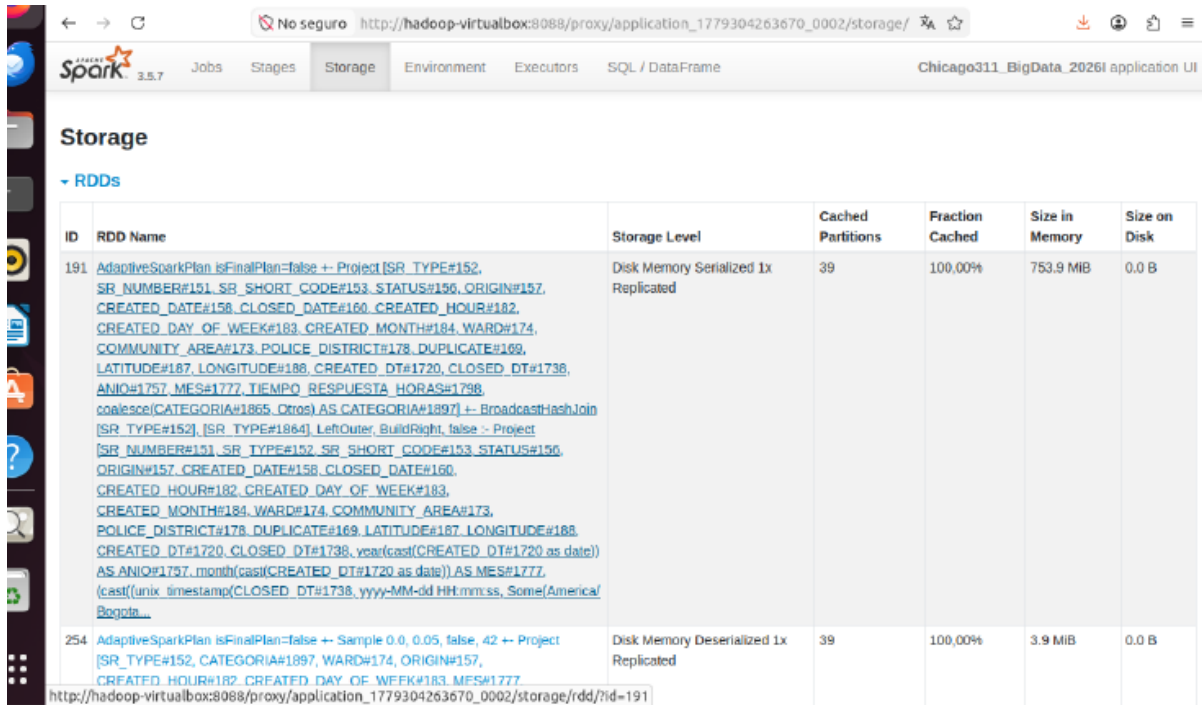
Miscellaneous Process

Show 20 entries Search:

Process ID	Address	Status	Cores	Logs
------------	---------	--------	-------	------

Pie de página: Detalle por executor: driver en hadoop-VirtualBox (4.3 MiB storage, 0 tasks de cómputo), executor 1 en nodo3:37891 con 19 RDD Blocks, 367.7 MiB / 1.6 GiB en memoria, 2 cores, 385 tasks completados, 42.7 GiB de input, 100.6 MiB shuffle read, 102.5 MiB shuffle write. Executor 2 en nodo2:42989 con 20 RDD Blocks, 393.2 MiB / 1.6 GiB en memoria, 2 cores, 408 tasks completados, 45.7 GiB de input, 111.9 MiB shuffle read, 110 MiB shuffle write. El balance casi simétrico entre nodo2 (408 tasks) y nodo3 (385 tasks) demuestra paralelismo efectivo y distribución equilibrada del trabajo — evidencia definitiva de ejecución distribuida real.

Captura 29 — Spark UI Storage: RDDs cacheados



ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
191	<code>AdaptiveSparkPlan isFinalPlan=false -- Project [SR_TYPE#152, SR_NUMBER#151, SR_SHORT_CODE#153, STATUS#156, ORIGIN#157, CREATED_DATE#158, CLOSED_DATE#160, CREATED_HOUR#182, CREATED_DAY_OF_WEEK#183, CREATED_MONTH#184, WARD#174, COMMUNITY_AREA#173, POLICE_DISTRICT#178, DUPLICATE#169, LATITUDE#187, LONGITUDE#188, CREATED_DT#1720, CLOSED_DT#1738, ANIO#1757, MES#1777, TIEMPO_RESPUESTA_HORAS#1798, coalesce(CATEGORIA#1865, Otros) AS CATEGORIA#1897] -- BroadcastHashJoin [SR_TYPE#152], [SR_TYPE#1864] LeftOuter, BuildRight, false -- Project [SR_NUMBER#151, SR_TYPE#152, SR_SHORT_CODE#153, STATUS#156, ORIGIN#157, CREATED_DATE#158, CLOSED_DATE#160, CREATED_HOUR#182, CREATED_DAY_OF_WEEK#183, CREATED_MONTH#184, WARD#174, COMMUNITY_AREA#173, POLICE_DISTRICT#178, DUPLICATE#169, LATITUDE#187, LONGITUDE#188, CREATED_DT#1720, CLOSED_DT#1738, year(cast(CREATED_DT#1720 as date)) AS ANIO#1757, month(cast(CREATED_DT#1720 as date)) AS MES#1777, (cast((unix_timestamp(CLOSED_DT#1738, yyyy-MM-dd HH:mm:ss, Some(America/Bogota_</code>	Disk Memory Serialized 1x Replicated	39	100.00%	753.9 MiB	0.0 B
254	<code>AdaptiveSparkPlan isFinalPlan=false -- Sample 0.0, 0.05, false, 42 -- Project [SR_TYPE#152, CATEGORIA#1897, WARD#174, ORIGIN#157, CREATED_HOUR#182, CREATED_DAY_OF_WEEK#183, MES#1777,</code>	Disk Memory Deserialized 1x Replicated	39	100.00%	3.9 MiB	0.0 B

Pie de página: Pestaña Storage del Spark UI mostrando los DataFrames cacheados con MEMORY_AND_DISK_SER. RDD ID 191: AdaptiveSparkPlan correspondiente al df_enriquecido (post broadcast join con categorías), Storage Level Disk Memory Serialized 1x Replicated, 39 Cached Partitions (100% cached), 753.9 MiB en memoria. RDD ID 254: df_ml_sample para el modelo ML, 39 particiones cacheadas al 100%, 3.9 MiB en memoria. Esta captura es la evidencia directa de la optimización mediante persistencia: los datos no se recomputan en cada acción, lo que reduce significativamente el tiempo total de ejecución del pipeline.

Sección H — Comandos de ejecución documentados

Secuencia de comandos ejecutados desde la terminal del nodo maestro hadoop-VirtualBox para levantar el clúster, ingestar el dataset, ejecutar el cuaderno y verificar el estado.

#	Descripción	Comando
1	Levantar HDFS (NameNode + DataNodes + Secondary NameNode)	<code>start-dfs.sh</code>
2	Levantar YARN (ResourceManager + NodeManagers)	<code>start-yarn.sh</code>
3	Levantar Spark History Server	<code>start-history-server.sh</code>
4	Instalar gdown para descargar el dataset desde Drive	<code>pip install gdown</code>
5	Descargar dataset al nodo maestro	<code>gdown 1bx70ng1IJxH-Srpyp1ebaLP5_aUcBi_v</code>
6	Crear directorio en HDFS	<code>hdfs dfs -mkdir -p /user/hadoop/chicago311</code>
7	Cargar dataset a HDFS con replicación 2	<code>hdfs dfs -D dfs.replication=2 -put 311_Service_Requests.csv /user/hadoop/chicago311/</code>
8	Verificar archivo en HDFS	<code>hdfs dfs -ls -h /user/hadoop/chicago311/</code>
9	Verificar bloques y estado de replicación	<code>hdfs fsck /user/hadoop/chicago311/311_Service_Requests.csv -files -blocks -locations</code>
10	Tamaño lógico y físico del directorio	<code>hdfs dfs -du -s -h /user/hadoop/chicago311/</code>
11	Eliminar archivo local tras carga a HDFS (requisito)	<code>rm 311_Service_Requests.csv</code>
12	Lanzar Jupyter Notebook	<code>jupyter notebook</code>

Sección I — Declaración obligatoria de uso de Inteligencia Artificial

En cumplimiento de la Comunicación rectoral n.º 004 del 4 de octubre de 2024 de la Universidad Externado de Colombia (Lineamientos para el Uso de Inteligencia Artificial), se declara a continuación el uso específico de herramientas de IA en cada componente del trabajo, conforme al numeral 2.1.2 del lineamiento institucional (categorías permitidas: Revisado con IA, Redactado con IA, Corregido con IA, Traducido con IA).

I.1 — Tabla de uso por componente

Componente	Categoría (Num. 2.1.2)	Herramienta	Detalle del aporte de IA
Selección del conjunto de datos	Sin uso de IA	—	Decisión 100% del equipo: identificación de la fuente oficial (Chicago Data Portal), validación de criterios de tamaño, fecha y cobertura.
Código PySpark	Revisado con IA	Claude (Anthropic)	Revisión de buenas prácticas en configuración de SparkSession, validación sintáctica de window functions y broadcast joins, sugerencias de optimización. La lógica algorítmica es del equipo.
Configuración del clúster	Sin uso de IA	—	Configuración manual de Hadoop + YARN + Spark sobre VirtualBox. Cálculo de parámetros según los recursos físicos disponibles realizado por el equipo.
Análisis e interpretación de resultados	Revisado con IA	Claude (Anthropic)	Revisión de redacción técnica. Las conclusiones, decisiones de modelado (RF vs LR, P75 como umbral, exclusión de columnas con >50% nulos) y priorización de recomendaciones son del equipo.
Informe escrito	Corregido con IA	Claude (Anthropic)	Asistencia en redacción ejecutiva y ajuste al formato exigido. El contenido sustantivo —métricas, hallazgos, recomendaciones— proviene del análisis del equipo.
Presentación y video	Sin uso de IA	—	Grabación y narración 100% del equipo. Sin uso de voces o avatares sintéticos.

I.2 — Contribución intelectual significativa del equipo (máx. 150 palabras)

Selección del conjunto de datos: el equipo identificó el dataset Chicago 311 Service Requests tras evaluar fuentes oficiales que cumplieran el rango 2–5 GB, actualización posterior a enero 2026 y cobertura temporal con datos de 2026, descartando alternativas pre-listadas. **Definición del objetivo:** la decisión de combinar análisis descriptivo (cuellos de botella por ward) con modelado predictivo (clasificación binaria de tardanza relativa al P75 del propio tipo) es propia, motivada por la utilidad operativa para una entidad municipal. **Justificación de optimizaciones:** la elección de shuffle.partitions=100 (vs. 200 por defecto), del StorageLevel MEMORY_AND_DISK_SER y de Random Forest sobre regresión logística se justifica técnicamente. **Interpretación de resultados:** la lectura crítica del AUC 0.81 como insumo de priorización (no infalible) y la identificación de redundancia espacial vía correlación son del equipo.

I.3 — Declaración firmada por los integrantes

Los abajo firmantes declaramos que hemos revisado el contenido final del presente trabajo y verificado la exactitud de la información generada con apoyo de Inteligencia Artificial, en cumplimiento del numeral 4.5.3 del lineamiento institucional. Asumimos plena responsabilidad intelectual y académica sobre los resultados, conclusiones y recomendaciones presentadas.

Nombre completo	Firma	Fecha
Allison Michelle Loango Rayo	 _____	26 / 05 / 2026
Angie Katherine Loango Rayo	 _____	26 / 05 / 2026
Rafael José Orozco Luquez	 _____	26 / 05 / 2026

Sección J — Referencias bibliográficas (APA 7ª edición)

- Apache Software Foundation. (2024). *Apache Hadoop Documentation — HDFS Architecture*.
<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- Apache Software Foundation. (2024). *Apache Spark 3.5 Documentation — SQL Performance Tuning*.
<https://spark.apache.org/docs/latest/sql-performance-tuning.html>
- Apache Software Foundation. (2024). *Apache Spark 3.5 Documentation — Running Spark on YARN*.
<https://spark.apache.org/docs/latest/running-on-yarn.html>
- Apache Software Foundation. (2024). *MLlib — Spark Machine Learning Library Guide*.
<https://spark.apache.org/docs/latest/ml-guide.html>
- City of Chicago. (2026). *311 Service Requests Dataset*. Chicago Data Portal.
<https://data.cityofchicago.org/Service-Requests/311-Service-Requests/v6vf-nfxy>
- Karau, H., & Warren, R. (2017). *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. O'Reilly Media.
- Universidad Externado de Colombia. (2024, 4 de octubre). *Lineamientos para el Uso de Inteligencia Artificial* (Comunicación rectoral n.º 004).
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.
<https://doi.org/10.1145/2934664>